Fall Semester 2025

Bachelor of Global Engineering (BoGE)



Robotics Bootcamp



Coordinators : Mohamad MROUÉ and Philippe BENABES

Instructors: Clara CIOCAN, Frédéric BOULANGER, Florent LOETE, Tony HANNA, Mohamad MROUÉ, and Philippe BENABES

Technical Support: Carles CATTY

CentraleSupélec | université PARIS-SACLAY

Course objective

This project-oriented course focuses on the practical application of **embedded systems**, using **Raspberry Pi (RPI) platform** to design and control **robotic systems**.





Robotic Systems

- A **robotic system** is a **specialized embedded system** that combines mechanical components, sensors, actuators, and software to perform autonomous or semi-autonomous tasks.
- As with other embedded systems, robotic systems feature a **control unit embedded** within the hardware, processing data from sensors and executing commands to actuators based on programmed algorithms.
- These systems are essential in fields like manufacturing, healthcare, and exploration, where they handle precise, repetitive, or complex tasks, often in environments that may be challenging or unsafe for human operators.

Embedded Systems

- **Embedded systems** are essential in our daily lives, integrated into various applications like smartphones, autonomous vehicles, household appliances, and medical equipment. They are at the core of modern robotics, enabling complex functionalities from basic control to advanced AI processing.
- An embedded system is a **standalone computing system** within a **physical device**, designed to **perform specific tasks autonomously**, **without constant human intervention**, and to operate under defined criteria.
- These systems are typically **compact**, **energy-efficient**, and capable to handle **real-time requirements**, meeting **strict performance** and **reliability standards**.

Embedded Systems – Applications

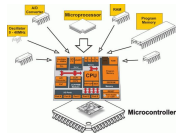
- **Robotics** and industrial automation
- Medical and Consumer Electronics
- Automotive and Transportation
- Aerospace and Defense
- Telecommunications
- Renewable energy





Embedded Systems Technologies for Robotic System Design

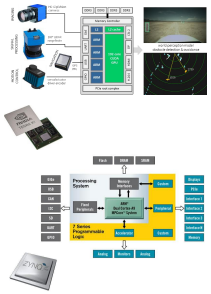
- **Microcontrollers** are integrated circuits that combine a CPU, memory, and peripherals on a single chip, like the **ATmega328P** used in **Arduino boards**.
  - They are ideal for simple, real-time control tasks such as motor control, sensor reading, and basic decision-making.
  - Microcontrollers are advantageous due to their low power consumption, cost-effectiveness, and ease of programming for specific, dedicated functions.
- **Microprocessors**, on the other hand, act as standalone CPUs and require external components for memory and I/O operations, as seen in the **ARM Cortex-A72** processor within the **Raspberry Pi 4**'s **Broadcom BCM2711 SoC**.
  - They are commonly used in more complex robotic systems requiring high processing power for multitasking and data-intensive tasks, making them suitable for applications that demand substantial computational capability.





Embedded Systems Technologies for Robotic System Design

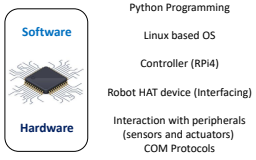
- A **System on Chip** (SoC) is a single chip that integrates a CPU, GPU, memory, and I/O interfaces, like the **NVIDIA Tegra X1** in the **NVIDIA Jetson**.
  - SoCs are widely used in robotics for handling complex tasks such as real-time image processing, AI algorithms, and multitasking.
  - Their high performance, efficient power usage, and ability to support advanced applications make them ideal for demanding robotic functions that require substantial processing power and multitasking capabilities.
- An **SoC-FPGA** combines the functionalities of an SoC with a **Field Programmable Gate Array** (FPGA), enabling both **software processing** and **customizable hardware acceleration**, as seen in the **Xilinx Zynq series**.
  - In robotics, SoC-FPGAs are valuable for applications needing **real-time processing**, **high-speed control loops**, and **flexible logic**, such as **sensor fusion** and **vision processing**.
  - This integration offers advantages like flexibility, **parallel processing**, and the capacity to **reconfigure hardware** to optimize specific robotic tasks.



Provided Robot Kit and Selected Technology

- The provided robot kit includes the following components:
  - A **Raspberry Pi 4 board**.
  - An ultrasonic sensor for obstacle detection.
  - Line-following sensors.
  - A servo motor to steer the front wheels.
  - DC motors to drive the car.
  - A camera for computer vision tasks.
  - Accelerometer and Gyroscope.
  - Two servo motors to adjust the robot's head, allowing it to point the camera and the ultrasonic sensor in specific directions.
- Possible features to implement:
  - **Line following**: Using sensors to follow a designated path.
  - **Obstacle avoidance**: Detecting and avoiding obstacles.
  - **Object recognition/tracking**: Utilizing the camera module to recognize and follow objects.
  - **Remote control**: Controlled via a web interface or a mobile app.





Course Outline and Organization

Wednesday Nov. 12th			
Duration	From	To	**Introduction**
1h	08:15	09:15	Course Objectives & Introduction
2h00	09:30	12:00	Introduction to Raspberry Pi, Robot HAT, Linux Command Line Interface, and Python Programming (Online Python IDE)
3h00	13:15	17:00	

Thursday Nov. 13th			
Duration	From	To	**Tutorials**
3h00	08:15	12:00	Configuring and Interfacing with Robot Peripherals., DC and Servo motors, GPIOs, LEDs, Ultrasonic module, Tracking module, MPU6050 module and Displaying Text on the OLED screen
3h00	13:15	17:00	

Friday Nov. 14th			
Duration	From	To	**Phase 1**
3h00	08:15	12:00	Line follower robot based on the tracking module and controlling DC and Servo motors (use OLED to display of actual system behavior)
3h00	13:15	17:00	

Monday Nov. 17th			
Duration	From	To	**Phase 1**
3h00	08:15	12:00	Line follower robot based on the tracking module and controlling DC and Servo motors (use OLED to display of actual system behavior)
3h00	13:15	17:00	

Wednesday Nov. 19th			
Duration	From	To	**Phase 2**
Canceled			Introduce the MPU6050 sensor and the ultrasonic sensor to detect and avoid possible obstacles
3h00	13:15	17:00	

Thursday Nov. 20th			
Duration	From	To	**Phase 2**
3h00	08:15	12:00	Introduce the MPU6050 sensor and the ultrasonic sensor to detect and avoid possible obstacles
3h00	13:15	17:00	

Friday Nov. 21st			
Duration	From	To	**Demonstration and Evaluation**
3h30	08:15	12:00	Final test and validation
3h30	13:15	17:00	Presentation and demonstration

Evaluation Method:
- **Tutorials & Tasks: 40 %**
- **Demonstration: 30 %**
- **Presentation & Q/A: 30 %**

4 Students per Team

Student Teams	Instructor
T01, T02, T03, T04, T05	Mohamad MROUE
T06, T07, T08, T09	Frédéric BOULANGER
T10, T11, T12, T13	Tony HANNA
T14, T15, T16, T17	Clara CIOCAN / Florent LOETE

Project Outline and Organization

- **Phase 1 – Line Following:** Program the robot to detect and follow visible line marked on the ground.
  - Different difficulty levels, such as **straight lines**, **circular paths**, **zigzags**, **sharp corners**, and **ascending and descending slopes** are introduced to evaluate the robot's performance.



- **Phase 2 – Obstacle Avoidance:** Enable the robot to detect, avoid and bypass obstacles along the path using the ultrasonic sensor.

- **Bonus:** Integrate the camera for advanced line and obstacle detection and recognition.

Robotic System Design Steps

1. **Concept and Requirements:**
   - Define the project's purpose (e.g., line following, obstacle avoidance) and list functional requirements
   - Identify hardware and software specifications (e.g., Raspberry Pi, motor drivers, sensors).

2. **Development Flow:**
   - Outline high-level and low-level designs, detailing architecture and module interactions.
   - Develop code for each feature (e.g., line following, remote control) using a modular approach.

3. **Testing and Deployment:**
   - **Unit Testing:** Validate each feature individually, refining components based on test results.
   - **Integration and System Testing:** Combine modules, test interactions, and ensure seamless functionality.
   - **Field Testing and Iteration:** Conduct real-world tests, gather feedback, and iterate to improve.
   - **Documentation and Deployment:** Create technical documentation, present a demo, and plan for system maintenance.

Project Management

- **Define Scope & Requirements:**
  - Set clear goals (e.g., line following, obstacle avoidance).
  - Identify hardware/software needs (e.g., sensors, Raspberry Pi).

- **Task Allocation & Timeline:**
  - Assign tasks based on team strengths.
  - Establish milestones and deadlines.

- **Modular Development:**
  - Develop components independently (e.g., line following, obstacle detection).
  - Schedule regular integration points for testing.

- **Testing & Iteration:**
  - Unit test each module, then integrate and system test.
  - Conduct field tests, gather feedback, and refine as needed.

- **Documentation & Communication:**
  - Maintain thorough documentation.
  - Schedule regular check-ins to ensure alignment and address challenges.

Presentation outline

1. Introduction to Python Programming
2. Introduction to Raspberry Pi and Robot HAT
3. Introduction to Linux Command Line Interface (CLI)
4. Configuring and Integrating Robot Peripherals (Part 1)
   - Controlling LED (Warm color and WS2812)
   - Controlling DC motors
5. Advanced Programming Techniques
6. Configuring and Integrating Robot Peripherals (Part 2)
   - Multi-Threaded Control of Warm color LED
   - Controlling Servo motors
   - Interfacing with the Ultrasonic module
   - Interfacing with the Tracking module
   - Reading data from the MPU6050 module
   - Displaying Text on the OLED screen

Presentation outline

1. **Introduction to Python Programming**
2. Introduction to Raspberry Pi and Robot HAT
3. Introduction to Linux Command Line Interface (CLI)
4. Configuring and Integrating Robot Peripherals (Part 1)
   - Controlling LED (Warm color and WS2812)
   - Controlling DC motors
5. Advanced Programming Techniques
6. Configuring and Integrating Robot Peripherals (Part 2)
   - Multi-Threaded Control of Warm color LED
   - Controlling Servo motors
   - Interfacing with the Ultrasonic module
   - Interfacing with the Tracking module
   - Reading data from the MPU6050 module
   - Displaying Text on the OLED screen

Introduction to Python Programming

- Python is a high-level, general-purpose programming language known for its simplicity and readability.
  - Created by Guido van Rossum in 1991, it supports multiple programming paradigms like procedural, object-oriented, and functional programming.
  - Python's clean and intuitive syntax makes it easy to learn and use, allowing developers to write complex code more efficiently.
    - This makes it suitable for both beginners and experienced programmers.

- Python is highly versatile, with applications in web development, data analysis, artificial intelligence, and automation.
  - It has a rich ecosystem of libraries and frameworks, and its large community provides extensive support and resources for developers.

Online Python Support and Tutorials

- Python.org (Official Python Documentation)
  - https://docs.python.org/3/
  - https://docs.python.org/3/reference/index.html (Language Reference)

- W3Schools (Built-in code editor for quick testing)
  - https://www.w3schools.com/python/default.asp

- Real Python (Focus on practical applications)
  - https://realpython.com/

- Online Python Integrated Development Environment (IDE)
  - https://www.online-python.com/
  - https://readypy.com/

Python – Operators

- Python supports a wide variety of operators which act like functions, i.e. they do something and return a value:
  - Arithmetic: `+` `-` `*` `/` `%` `**`
  - Logical: `and` `or` `not`
  - Comparison: `>` `<` `>=` `<=` `!=` `==`
  - Assignment: `=`
  - Bitwise: `&` `|` `~` `^` `>>` `<<`
  - Identity: `is` `is not`
  - Membership: `in` `not in`

Python – Arithmetic Operators

- Arithmetic operators are used to perform basic mathematical operations such as addition, subtraction, multiplication, division, etc.

Operator	Description	Example
`+`	Addition	`x + y`
`-`	Subtraction	`x - y`
`*`	Multiplication	`x * y`
`/`	Division	`x / y`
`%`	Modulus (remainder)	`x % y`
`**`	Exponentiation	`x ** y` (x raised to the power y)
`//`	Floor Division	`x // y` (division without remainder)

```
x = 10
y = 3
print(x + y)   # Output: 13
print(x / y)   # Output: 3.3333
print(x // y)  # Output: 3 (floor division)
```

Python – Logical Operators

- Logical operators are used to combine conditional statements or Boolean expressions.

Operator	Description	Example
`and`	True if both conditions are true	`x and y`
`or`	True if at least one condition is true	`x or y`
`not`	Reverses the result of the condition	`not x`

```
x = True
y = False
print(x and y)      # Output: False
print(x or y)       # Output: True
print(not x)        # Output: False
```


Python – Comparison Operators

- Comparison operators are used to compare two values. The result is always a Boolean value (True or False).

Operator	Description	Example
`==`	Equal to	`x == y`
`!=`	Not equal to	`x != y`
`>`	Greater than	`x > y`
`<`	Less than	`x < y`
`>=`	Greater than or equal to	`x >= y`
`<=`	Less than or equal to	`x <= y`

```
x = 10
y = 20
print(x == y)  # Output: False
print(x < y)   # Output: True
```


Python – Assignment Operators

- Assignment operators are used to assign values to variables.

Operator	Description	Example
`=`	Assigns a value to a variable	`x = 5`
`+=`	Adds and assigns	`x += 3` (same as `x = x + 3`)
`-=`	Subtracts and assigns	`x -= 3`
`*=`	Multiplies and assigns	`x *= 3`
`/=`	Divides and assigns	`x /= 3`
`**=`	Raises to power and assigns	`x **= 3`
`//=`	Floor divides and assigns	`x //= 3`

```
x = 10
x += 5        # x is now 15
print(x)      # Output: 15
```


Python – Bitwise Operators

- Bitwise operators work on binary numbers and perform operations bit by bit.

Operator	Description	Example
`&`	Bitwise AND	`x & y`
`|`	Bitwise OR	`x | y`
`^`	Bitwise XOR	`x ^ y`
`~`	Bitwise NOT	`~x`
`<<`	Left Shift	`x << y`
`>>`	Right Shift	`x >> y`

```
x = 5              # In binary: 101
y = 3              # In binary: 011
print(x & y)       # Output: 1 (binary 001)
```


Python – Identity Operators

- Identity operators are used to compare the memory locations of two objects.

Operator	Description	Example
`is`	Returns True if both variables point to the same object	`x is y`
`is not`	Returns True if both variables point to different objects	`x is not y`

```
x = [1, 2, 3]
y = [1, 2, 3]
print(x is y)  # Output: False (because they are different objects in memory)
print(x == y)  # Output: True (because the values are the same)
```


Python – Membership Operators

- Membership operators are used to test whether a value or variable is found in a sequence (such as a list, tuple, string, etc.).

Operator	Description	Example
`in`	Returns True if a value is found in a sequence	`x in y`
`not in`	Returns True if a value is not found in a sequence	`x not in y`

```
x = "hello"
print("h" in x)            # Output: True
print("z" not in x)        # Output: True
```


Python – Comments

- **Single-line comments**: Use the hash symbol (#) to create a single-line comment. Everything after the # on the same line will be ignored by the Python interpreter.

```
# This is a comment
# that spans multiple lines
print("Hello, World!")
```

- **Multi-line comments**: enclosed in triple quotes ''':

```
"""
This is a multi-line comment.
It can span multiple lines.
"""
print("Hello, World!")
```


Python – Variables and Data Types

- A **variable** in Python is used to store data values.
  - It is like a labeled container that holds information which can be referenced and manipulated in your program.
  - Variables in Python are dynamically typed, meaning you don't need to explicitly declare their data type; Python automatically determines the type based on the value assigned. You can assign values to variables using the assignment operator **=**.
  - Variable names are case-sensitive. They can include letters, numbers, and underscores, but must start with a letter or an underscore. They cannot contain spaces or start with a number.

Python – Variables and Data Types

```
# Assigning a value to a variable
name = "Alice"          # A string value
age = 25                # An integer value
height = 5.6            # A float value
is_student = True       # A boolean value

# Output the values
print(name)             # Output: Alice
print(age)              # Output: 25
print(height)           # Output: 5.6
print(is_student)       # Output: True
```

Python – Variables and Data Types

```
# Assigning the same value to multiple variables
a = b = c = 10

# Assigning different values to different variables in one line
x, y, z = 5, 10, 15
r, s, t = 1, 3.57, 'Alice'

print(a, b, c)          # Output:   10    10    10
print(x, y, z)          # Output:   5     10    15
print(r, s, t)          # Output:   1     3.57  Alice
```


Python – Variables and Data Types

- In Python, data types define the kind of data a variable can hold.
  - Since Python is dynamically typed, the type of a variable is determined automatically based on the value assigned to it.
  - You can use the `type()` function to check the data type of a variable.

```
x = 42
print(type(x))  # Output: <class 'int'>

y = 3.14
print(type(y))  # Output: <class 'float'>

z = "Hello"
print(type(z))  # Output: <class 'str'>
```


Python – Numeric Types

- Python supports three numeric types: integers, floating-point numbers, and complex numbers.
  - **Integer** (`int`): Represents whole numbers, both positive and negative.

```
x = 10  # int
y = -5  # int
```

  - **Floating-point** (`float`): Represents decimal numbers.

```
pi = 3.14               # float
temperature = -5.5      # float
```

  - **Complex** (`complex`): Represents complex numbers with a real and imaginary part.

```
z = 3 + 4j        # complex
t = complex(3,4)  # complex
```


Python – String (`str`)

- Strings are sequences of characters, enclosed in single (') or double (") quotes.

```
greeting = "Hello, World!"       # str
name = 'Alice'                   # str
```

- You can also use triple quotes for multi-line strings:

```
message = """This is
a multi-line
string."""
```


Python – String (`str`)

- Strings are immutable, meaning their values cannot be changed after creation.

```
greeting = "Hello"
greeting[0] = "h"            # This will raise an error
```

- The above code will result in an error because you cannot change individual characters in the string. However, you can reassign the variable to a new string:

```
text = "Python"
text = "Java"  # This is valid, but a new string "Java" is created
```


Python – String (`str`)

- Python provides many **built-in string functions** that help in performing operations on strings, such as modifying, searching, formatting, and analyzing string content.

```
text = "hello"
print(text.upper())  # Output: "HELLO"

text = "hello world"
print(text.capitalize())  # Output: "Hello world"

text = "hello world"
print(text.find("world"))  # Output: 6
print(text.find("Python"))  # Output: -1 (not found)
```

```
text = "HELLO"
print(text.lower())  # Output: "hello"

text = "hello world"
print(text.title())  # Output: "Hello World"

text = "hello world"
print(text.index("world"))  # Output: 6
# Raises ValueError if substring is not found
```


Python – String (`str`)

- The concatenation operator (+) is used to combine two or more strings into a single string.

```
str1 = "Hello"
str2 = "World"
result = str1 + " " + str2
# Concatenates the two strings with a space in between
print(result)  # Output: "Hello World"
```

- You can append a string to another string using the += operator.

```
# Initial string
greeting = "Hello"
# Appending to the string using +=
greeting += ", World!"
print(greeting)
# Output: "Hello, World!"
```


Python – String (`str`)

- The repetition operator (*) is used to repeat a string a specified number of times.
  - Syntax: `str * n` where n is an integer that specifies how many times to repeat the string.
  - Example:

```
text = "Python! "
result = text * 3  # Repeats the string 3 times
print(result)  # Output: "Python! Python! Python! "
```


Python – Boolean (`bool`)

- Booleans represent two values: **`True`** or **`False`**.
  - They are often used in conditions and logical operations.

```
is_active = True       # bool
has_error = False      # bool
```

Python – Sequence Types

- Python offers several types to store sequences of items, such as lists, tuples, and ranges.
  - **List** (`list`): A mutable, ordered collection of items. Lists can contain elements of different types.

```
numbers = [1, 2, 3, 4, 5]           # list of integers
mixed = [1, "apple", 3.14, True]    # list with mixed types
```

  - **Tuple** (`tuple`): An immutable, ordered collection of items. Once created, the elements of a tuple cannot be changed.

```
coordinates = (10, 20)                  # tuple
fruits = ("apple", "banana", "cherry")  # tuple of strings
```

  - **Range** (`range`): Represents a sequence of numbers, often used in loops.

```
numbers = range(1, 6)   # range from 1 to 5
```


Python – Dictionary (dict)

- Dictionaries are another basic Python data type. They are unordered collections of ***key-value*** pairs.
  - They are mutable, meaning you can change, add, or remove key-value pairs.

```
x = {'a_key':55, 100:'a_value', 4.1:[5,6,7]}
```

  - Accessing a value:

```
print(x['a_key'])     # Output: 55
print(x[100])         # Output: 'a_value'
```


Python – Dictionary (dict)

- Example:

```
x={}
print(x)            # Output: {}

x[4] = 'value_b'
print(x)            # Output: {4: 'value_b'}

x['key_a'] = 55
print(x)            # Output: {4: 'value_b', 'key_a': 55}

x[103] = 'dict'
print(x)            # Output: {4: 'value_b', 'key_a': 55, 103: 'dict'}

del x['key_a']      # deletes a key/value
print(x)            # Output: {4: 'value', 103: 'dict'}
```


`len()` Function in Python

- The `len()` function in Python is used to determine the length (the number of items) of an object.
- It returns an integer representing the count of elements in various types of objects like strings, lists, tuples, dictionaries, and other iterable objects.

```
text = "Hello, World!"
print(len(text))  # Output: 13

numbers = [1, 2, 3, 4, 5]
print(len(numbers))  # Output: 5

my_dict = {"name": "Alice", "age": 25, "city": "New York"}
print(len(my_dict))  # Output: 3

x = 100
print(len(x))  # TypeError: object of type 'int' has no len()
```

```
my_tuple = (10, 20, 30)
print(len(my_tuple))  # Output: 3
```


Python – if statement

- In Python, conditional statements allow you to execute certain blocks of code based on whether a condition is true or false. The primary keywords for conditional statements are `if`, `elif`, and `else`.
  - `elif` and `else` are not required – used to chain together multiple conditional statements or provide a default case.

```
if condition1:
    # Code block for condition1
elif condition2:
    # Code block for condition2
else:
    # Code block if none of the above conditions are true
```


Python – if else

- Python uses whitespace (spaces or tabs) to define code blocks.
  - Code blocks are logical groupings of commands. They are always preceded by a colon :
  - Python knows a code block has ended when the indentation is removed.
- Example 1:

```
x = 8

if x > 10:
    print("x is greater than 10")
elif x == 8:
    print("x is equal to 8")
else:
    print("x is less than 10 but not 8")
# Output: "x is equal to 8"
```

A code block

Another code block

Python – Nested if statement

- Code blocks can be nested inside others therefore `if-elif-else` statements can be freely nested within others.
- Example 2:

```
age = 20
has_ID = True

if age >= 18:
    if has_ID:
        print("You are eligible to vote.")
    else:
        print("You need an ID to vote.")
else:
    print("You are not old enough to vote.")
```


Python – Lists

- A **list** in Python is a versatile and ordered collection of items, which can hold elements of different data types such as integers, floats, strings, or even other lists.
- **Lists** are mutable, meaning their contents can be modified (you can add, remove, or change elements).
- **Lists** are one of the most commonly used data structures in Python due to their flexibility.

Python – Creating a List and Accessing Elements

- Lists are defined by placing elements inside square brackets [], separated by commas.

```
# Examples of lists
numbers = [1, 2, 3, 4, 5]                  # A list of integers
fruits = ["apple", "banana", "cherry"]     # A list of strings
mixed = [1, "hello", 3.14, True]           # A list with mixed data types
empty_list = []                            # An empty list
```

- You can access individual elements in a list using indexing. Indexing starts from 0 (first element) and goes up to n-1 (last element), where n is the length of the list.

```
# Accessing elements
fruits = ["apple", "banana", "cherry"]
print(fruits[0])      # Output: apple
print(fruits[2])      # Output: cherry
```

Python – Modifying Lists

- Since lists are mutable, you can change, add, or remove elements.
  - Changing elements

```
fruits = ["apple", "banana", "cherry"]
fruits[1] = "orange"          # Changing the second element
print(fruits)                 # Output: ['apple', 'orange', 'cherry']
```

  - Adding Elements:
    - `append()`: Adds an element to the end of the list.

```
fruits.append("grape")
print(fruits)  # Output: ['apple', 'orange', 'cherry', 'grape']
```

    - `insert()`: Adds an element at a specified position.

```
fruits.insert(1, "banana")  # Inserts 'banana' at index 1
print(fruits)  # Output: ['apple', 'banana', 'orange', 'cherry', 'grape']
```


Python – Modifying Lists

  - Removing Elements:
    - `remove()`: Removes the first occurrence of a specified element.

```
fruits.remove("orange")
print(fruits)  # Output: ['apple', 'banana', 'cherry', 'grape']
```

    - `pop()`: Removes and returns the element at the specified index. If no index is provided, it removes the last element.

```
fruits.pop(2)
print(fruits)  # Output: ['apple', 'banana', 'grape']
```

    - Lists can have their elements overwritten or deleted (with the `del`) command.

```
List: x=['a', 'b', 'c', 'd' ,'e']
x[0] = -3.14          # x is now [-3.14, 'b', 'c', 'd', 'e']
del x[-1]             # x is now [-3.14, 'b', 'c', 'd']
```


Python – List Slicing

- You can extract a subset of a list using slicing. Slicing allows you to create a new list from a part of an existing list.

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
subset = numbers[2:6]  # Slices from index 2 to 5 (6 is excluded)
print(subset)  # Output: [2, 3, 4, 5]


# Omitting start or end
print(numbers[:4])  # Output: [0, 1, 2, 3] (from start to index 3)
print(numbers[5:])  # Output: [5, 6, 7, 8, 9] (from index 5 to end)
```


Python – Common List Methods

- Python provides several built-in methods to work with lists:
  - `len()`: Returns the number of elements in the list.

```
print(len(fruits))            # Output: 3
```

  - `sort()`: Sorts the list in ascending order.

```
numbers = [4, 2, 9, 1, 5]
numbers.sort()
print(numbers)                # Output: [1, 2, 4, 5, 9]
```

  - `reverse()`: Reverses the order of elements in the list.

```
numbers.reverse()
print(numbers)                # Output: [9, 5, 4, 2, 1]
```

  - `count()`: Returns the number of times an element appears in the list.

```
fruits = ["apple", "banana", "apple", "cherry"]
print(fruits.count("apple"))  # Output: 2
```


Python – Using the Repetition Operator with Lists

- The repetition operator (*) can be used with lists in Python. It allows you to repeat the elements of a list a specified number of times, creating a new list with the repeated elements.
  - Syntax:

```
new_list = original_list * n
```

    - `original_list`: The list you want to repeat.
    - `n`: The number of times you want to repeat the list.

```
# Repetition with a list
numbers = [1, 2, 3]
repeated_numbers = numbers * 3
print(repeated_numbers)
# Output: [1, 2, 3, 1, 2, 3, 1, 2, 3]
```

```
pattern = ["A", "B", "C"]
repeated_pattern = pattern * 2
print(repeated_pattern)
# Output: ['A', 'B', 'C', 'A', 'B', 'C']
```


Python – Using the Repetition Operator with Lists

- The repetition operator (*) can be used with lists in Python. It allows you to repeat the elements of a list a specified number of times, creating a new list with the repeated elements.
  - Syntax:

```
new_list = original_list * n
```

    - `original_list`: The list you want to repeat.
    - `n`: The number of times you want to repeat the list.

```
list_of_lists = [[1, 2]] * 3
print(list_of_lists)
# Output: [[1, 2], [1, 2], [1, 2]]

# Modifying one sublist affects all because they share the same reference
list_of_lists[0][0] = 99
print(list_of_lists)
# Output: [[99, 2], [99, 2], [99, 2]]
```


Python – Variables and Memory Locations

- When you create a list and assign it to a variable, the variable acts as a reference (or a label) to the memory location where the list is stored.
- If you assign this list to another variable, you are not creating a new copy of the list but; instead, both variables refer to the same original list.

```
# Create a list and assign it to variable 'a'
a = [1, 2, 3]

# Assign 'a' to another variable 'b'
b = a

# Modify the list using 'b'
b.append(4)

# Output both lists
print(a)    # Output: [1, 2, 3, 4]
print(b)    # Output: [1, 2, 3, 4]
```


Python – Variables and Memory Locations

- Creating a True Copy of a List
  - If you want to create a completely new list with the same content (a copy) instead of a reference to the same memory location, you can use the `copy()` method or list slicing.
  - List Assignment (`b = a`): Creates a new reference to the same memory location. Changes made through either variable (a or b) will affect the same list.
  - List Copy (`b = a.copy()`, `b = list(a)` or `b = a[:]`): Creates a new list in a different memory location. Changes to b will not affect a.

```
# Use copy() to create an actual copy of the list
a = [1, 2, 3]
b = a.copy()  # Creates a new list with the same elements

# Modify the copied list
b.append(4)

# Output both lists
print(a)  # Output: [1, 2, 3]  (original list is unchanged)
print(b)  # Output: [1, 2, 3, 4]  (new list with changes)
```


Python – Loops

- Loops in Python are constructs that allow you to execute a block of code repeatedly based on a condition or a set number of iterations.
- They are fundamental for performing repetitive tasks, iterating over collections (like lists or strings), and automating processes.
- Python provides two main types of loops: `for` loops and `while` loops.

Loop Type	Use Case	Example
`for` loop	Iterate over a sequence or range	`for item in list:`
`while` loop	Repeat until a condition is false	`while condition:`
`break`	Exit the loop immediately	`if condition: break`
`continue`	Skip the current iteration and continue	`if condition: continue`
`else`	Run if the loop finishes normally	`for ... else:` or `while ... else:`

Python – for Loop

- The for loop is used to iterate over a sequence (such as a list, tuple, string, or range) and execute a block of code for each element in the sequence.

  – Syntax:

```
for variable in sequence:
# Code block to execute
```

  – Example:

```
# Iterating over a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
# Output:
# apple
# banana
# cherry
```


Python – while Loop

- The while loop continues to execute a block of code as long as a given condition is True. It is often used when the number of iterations is not known beforehand.

- Syntax:

```
while condition:
    # Code block to execute
```

- Example:

```
# Loop until a condition is false
count = 0
while count < 5:
    print("Count:", count)
    count += 1
```



```
# Output:
# Count: 0
# Count: 1
# Count: 2
# Count: 3
# Count: 4
```


Python – Control Statements in Loops

- `break` Statement: Exits the loop immediately, even if the condition is still True.

```
# Break the loop when a condition is met
for number in range(10):
    if number == 5:
        break
    print(number)
```

```
# Output:
# 0
# 1
# 2
# 3
# 4
```


Python – Control Statements in Loops

- `continue` Statement: Skips the current iteration and moves to the next one.

```
# Skip printing number 5
for number in range(10):
    if number == 5:
        continue
    print(number)
```

```
# Output:
# 0
# 1
# 2
# 3
# 4
# 6
# 7
# 8
# 9
```


Python – Control Statements in Loops

- In Python, both for and while loops can have an optional `else` clause. The `else` block executes only if the loop completes normally (i.e., without a break).

```
# Using else with a for loop
for number in range(5):
    print(number)
else:
    print("Loop completed successfully.")

# Output:
# 0
# 1
# 2
# 3
# 4
# Loop completed successfully.
```


Python – Control Statements in Loops

- In Python, both for and while loops can have an optional `else` clause. The `else` block executes only if the loop completes normally (i.e., without a break).

```
# A for loop with else
numbers = [1, 2, 3, 4, 5]
for number in numbers:
    if number == 3:
        break
    print(number)
else:
    print("Loop completed without encountering a 3.")

# Output:
# 1
# 2
```


Python – Control Statements in Loops

- In Python, both for and while loops can have an optional `else` clause. The `else` block executes only if the loop completes normally (i.e., without a break).

```
count = 0
while count < 5:
    if count == 3:
        break
    print("Count is:", count)
    count += 1
else:
    print("Count reached 5.")
```

```
# Output:
# Count is: 0
# Count is: 1
# Count is: 2
```


Exercise 1: Build a Shopping Cart Program

- In this exercise, you will create a simple shopping cart program where a user can add items, view the cart, and remove items using Python lists and loops.
- Instructions:
  - Initialize the Cart: Start with an empty list called cart to hold the items the user adds.
  - Display Options: Present the user with the following options in a loop:
    - "1 - Add item to cart"
    - "2 - View cart"
    - "3 - Remove item from cart"
    - "4 - Quit"
  - Add Items:
    - If the user selects option 1, ask to enter the name of the item to add.
    - Append the item to the cart list.
    - Display a message confirming the item has been added.

Exercise 1: Build a Shopping Cart Program

  - View Cart:
    - If the user selects option 2, use a for loop to display each item in the cart.
    - If the cart is empty, display a message saying, "Your cart is empty."
  - Remove Items:
    - If the user selects option 3, ask him for the name of the item to remove.
    - Use a conditional statement to check if the item is in the cart.
    - If the item is found, remove it and confirm with a message.
    - If the item isn't in the cart, display a message saying, "Item not found in cart."
  - Quit the Program:
    - If the user selects option 4, exit the loop and display an end of program message.
  - Validation:
    - Ensure the user enters a valid option (1, 2, 3, or 4). If an invalid option is selected, prompt the user to try again.

Python – The range() function

- The `range()` function in Python is used to generate a sequence of numbers. It is commonly used in for loops to iterate a specified number of times.

  – Syntax: `range(start, stop, step)`

  – Parameters:
  - `start` (optional): The starting number of the sequence (inclusive). Default is 0.
  - `stop` (required): The ending number of the sequence (exclusive). The sequence will go up to, but not include, this number.
  - `step` (optional): The difference between each number in the sequence. Default is 1.

Python – The range() function

- Examples:

```
# Iterating from 0 to 4
for i in range(5):
    print(i)

# Output:
# 0
# 1
# 2
# 3
# 4
```

```
# Iterating from 2 to 5
for i in range(2, 6):
    print(i)

# Output:
# 2
# 3
# 4
# 5
```

```
# Iterating from 2 to 10
# with a step of 2
for i in range(2, 11, 2):
    print(i)
# Output:
# 2
# 4
# 6
# 8
# 10
```


Python – The range() function

- Examples:

```
# Print every third number
# from 0 to 15

for i in range(0, 16, 3):
    print(i)

# Output:
# 0
# 3
# 6
# 9
# 12
# 15
```

```
# Reverse countdown

for i in range(5, 0, -1):
    print(i)

# Output:
# 5
# 4
# 3
# 2
# 1
```

```
# Converting range to a list

numbers = list(range(5))
print(numbers)

# Output: [0, 1, 2, 3, 4]
```


Python – The range() function

- Examples:

```
# Iterating over a list using indices
# Using an f-string to format the output

fruits = ["apple", "banana", "cherry"]
for index in range(len(fruits)):
    print(f"Fruit {index}: {fruits[index]}")

# Output:
# Fruit 0: apple
# Fruit 1: banana
# Fruit 2: cherry
```

```
# Sum of the first 10 numbers

total = 0
for i in range(1, 11):
    total += i
print("Sum of numbers from 1 to 10 is:", total)

# Output: Sum of numbers from 1 to 10 is: 55
```

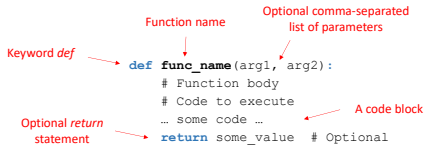

Python – Functions

- A function in Python is a block of reusable code that performs a specific task. Functions help organize code, make it modular, and avoid repetition.
  – You can define your own functions using the `def` keyword, or use Python's built-in functions.
  – Functions can accept inputs (called parameters), perform operations, and return outputs.

  – Why Use Functions?
  - **Modularity**: Break down complex problems into smaller, manageable parts.
  - **Reusability**: Write code once and use it multiple times.
  - **Readability**: Well-defined functions make the code easier to read and understand.
  - **Maintenance**: Easier to maintain and update code.

Python – Functions

- Defining a Function



  – The return value can be any Python type.
  – If the return statement is omitted a special none value is still returned.
  – The arguments are optional but the parentheses are required!
  – Functions must be defined before they can be called.

Python – Functions

- Function call syntax:

```
A = some_func()      # return a value
Another_func()       # ignore return value or nothing returned
b,c = multiple_vals(x,y,z)        # return multiple values
```

- Example:

```
# Function definition
def greet():
    print("Hello, World!")

# Function call
greet()
# Output: Hello, World!
```


Python – Functions – Parameters and Arguments

- Functions can accept inputs called **parameters**. When you call the function, you provide **arguments** that match these parameters.

```
# Function with parameters
def greet(name):
    print(f"Hello, {name}!")

# Function call with argument
greet("Alice")
# Output: Hello, Alice!
```


Python – Functions – Return Statement

- Functions can return a result using the `return` keyword. The `return` statement ends the function and sends a value back to the caller.

```
# Function with a return value
def add(a, b):
    return a + b

# Storing the result of the function call
result = add(3, 5)
print(result)
# Output: 8
```

Python – Types of Functions

- Built-in Functions: Functions that are part of Python's standard library, such as `print()`, `len()`, `sum()`, etc.

```
numbers = [1, 2, 3, 4, 5]
print(sum(numbers))  # Output: 15
```

- User-defined Functions: Functions that you create using the `def` keyword.

```
def multiply(a, b):
    return a * b

print(multiply(4, 5))  # Output: 20
```


Python – Functions – Default Parameters

- You can provide default values to parameters. If no argument is given when calling the function, the default value is used.

```
# Function with a default parameter
def greet(name="Guest"):
    print(f"Hello, {name}!")

# Calling without an argument
greet()  # Output: Hello, Guest!

# Calling with an argument
greet("Bob")  # Output: Hello, Bob!
```


Python – Functions – Keyword Arguments

- When calling a function, you can specify arguments by their parameter names using **keyword arguments**. This makes the code more readable and avoids errors.

```
def describe_person(name, age, city):
    print(f"{name} is {age} years old and lives in {city}.")

# Using keyword arguments
describe_person(name="Alice", age=30, city="New York")
# Output: Alice is 30 years old and lives in New York.
```


Python – Scope of Variables in Functions

- Variables defined inside a function are local to that function and cannot be accessed outside. Variables defined outside functions are global.

```
def local_scope():
    x = 10  # Local variable
    print(x)

local_scope()  # Output: 10
print(x)  # Error: x is not defined (outside the function)
```


Python – Functions

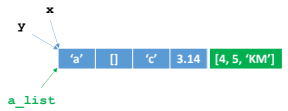
- Example:

```
x = ['a', [], 'c', 3.14]
y = x # y points to the same list as x

def my_func(a_list):
# modifies the list in the calling routine!
    a_list.append([4, 5, 'KM'])

print(y)
# Output : ['a', [], 'c', 3.14]

my_func(x) # x and a_list inside the function are the same list!
print(y)
# Output : ['a', [], 'c', 3.14, [4,5,'KM']]
```





Python – Functions

- If possible, pre-allocate a list or other large data structures and re-use it.
  - i.e. allocate outside the function call and send it as a function argument.
  - Remember function arguments are just references so they don't copy large data structures.

```
my_lst = 1000 * [0]
for i in range(1000):
    my_lst[i] = some_func()

# that is better than:
my_lst = []
for i in range(1000):
    my_lst.append(some_func())
```


Python – Tuples

- A tuple in Python is an ordered, immutable collection of elements.
  - Tuples are similar to lists in that they can hold multiple items, but unlike lists, the elements in a tuple cannot be modified once it is created.
  - Tuples are useful for storing data that should not change throughout the program.
  - Tuples are often used to store related pieces of information together.

```
# Creating a tuple with multiple elements
my_tuple = (1, 2, 3, 4)
print(my_tuple)  # Output: (1, 2, 3, 4)

# Tuple with mixed data types
mixed_tuple = ("apple", 10, 3.14, True)
print(mixed_tuple)  # Output: ('apple', 10, 3.14, True)

# Creating a tuple without parentheses (parentheses are optional)
another_tuple = 1, 2, 3
print(another_tuple)  # Output: (1, 2, 3)
```

```
# Unpacking a tuple
my_tuple = (10, 20, 30)
a, b, c = my_tuple
print(a)  # Output: 10
print(b)  # Output: 20
print(c)  # Output: 30
```


Python – Tuples

- Accessing Tuple Elements

```
# Accessing elements by index
my_tuple = (10, 20, 30, 40)
print(my_tuple[0])  # Output: 10
print(my_tuple[2])  # Output: 30

# Slicing a tuple
print(my_tuple[1:3])  # Output: (20, 30)

# Accessing the last element using negative indexing
print(my_tuple[-1])  # Output: 40

# Unpacking a tuple
my_tuple = (10, 20, 30)
a, b, c = my_tuple
print(a)  # Output: 10
print(b)  # Output: 20
print(c)  # Output: 30
```

```
def min_max(x):
    ''' Return the maximum
    and minimum values
    of x '''
    minval = min(x)
    maxval = max(x)
    # a tuple return...
    return minval, maxval

a = [10, 4, -2, 32.1, 11]
val = min_max(a)
min_a = val[0]
max_a = val[1]

# Or, easier...
min_a, max_a = min_max(a)
```


Python – Tuples

- Operations with Tuples
  - Concatenation

```
tuple1 = (1, 2)
tuple2 = (3, 4)
result = tuple1 + tuple2
print(result)  # Output: (1, 2, 3, 4)
```

  - Repetition

```
my_tuple = ("hello",) * 3
print(my_tuple)  # Output: ('hello', 'hello', 'hello')
```

  - Length

```
my_tuple = (10, 20, 30)
print(len(my_tuple))  # Output: 3
```

Python – Modules

- A **module** in Python is a file containing Python code that defines functions, classes, and variables, which can be reused in other Python programs.
- Modules are a way to organize and reuse code, promoting modularity and code separation.
- Modules can be:
  - Built-in Modules: These come pre-installed with Python (e.g., `math`, `os`, `sys`).
  - User-defined Modules: These are modules that you create yourself in separate `.py` files.
  - Third-party Modules: These are additional modules that you can install using package managers like `pip` (e.g., `requests`, `numpy`, `pandas`).

Python – Modules

- Creating and Using a Module
  - To create a module, simply save your code in a file with a .py extension. You can then use the import keyword to include the module in another program.

```
# greetings.py
def say_hello(name):
    print(f"Hello, {name}!")

def say_goodbye(name):
    print(f"Goodbye, {name}!")
```

```
# main.py
import greetings

greetings.say_hello("Alice")
# Output: Hello, Alice!
greetings.say_goodbye("Alice")
# Output: Goodbye, Alice!
```


Python – Modules

- Importing Specific Functions or Variables
  - Instead of importing the entire module, you can import specific functions or variables using the `from ... import ...` syntax.

```
# Importing specific functions from the module
from greetings import say_hello

say_hello("Bob")  # Output: Hello, Bob!

# Note: say_goodbye() is not imported and will not be available here
```

- Renaming a Module or Function with `as`
  - You can use the `as` keyword to give a module or function a different name (alias) when importing:

```
import greetings as greet
greet.say_hello("Charlie")  # Output: Hello, Charlie!
```


Python – Modules

- Built-in Modules Example
  - Python provides many built-in modules that you can use directly. Here are a couple of examples:

```
import math
print(math.sqrt(16))    # Output: 4.0
print(math.pi)          # Output: 3.141592653589793

import random
print(random.randint(1, 10))  # Output: A random number between 1 and 10
```


Exercise 2: Array Analysis

- You are given an array with a random number of elements. If this array contains between 3 and 100 integers, your task is to return the original array, the array sorted in both ascending and descending order, as well as display the minimum, maximum, mean, and standard deviation of the array's values.
- Steps:
  1. Create a function that takes a non-empty array of integers as input and returns the smallest value along with its index. Test this function.
  2. Create a function that takes a non-empty array of integers as input and returns the largest value. Test this function.
  3. Create a function that takes a non-empty array of integers as input and returns the array sorted in ascending order. Add a parameter called "order" so that if it is set to "asc", the output is the array sorted in ascending order; if it is "desc", the output is the array sorted in descending order. Test this function.

Exercise 2: Array Analysis

4. Create a function that takes a non-empty array of integers as input and returns the mean and standard deviation of the array's values. Test this function.
5. Create a function that validates any input to ensure it is a non-empty array of integers with a length between 3 and 100. This function should return a boolean value indicating whether the input is valid. Test this function.
6. Using the functions created above, write a main function that performs the desired operations, with appropriate error handling and informative error messages.

- Test the main function with the following inputs:

```
a = [1, 2, 3, 4]
b = [2, 2, 2, 2, 2, 2]
c = []
d = ['Problem', 'list', 1, 2, 3]
```

```
e = [1, 3.14, 5, 7, 10]
f = "Hello World"
h = 12
g = 44
```

```
i = (10, 20, 47)
j = range(1, 251, 2)
k = range(-50, 50, 1)
```

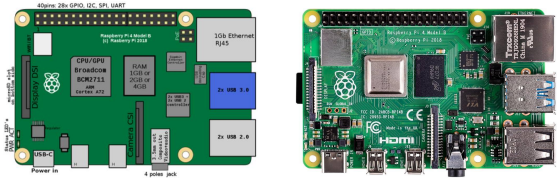
- BONUS: Aim to make each function robust, handling any possible errors gracefully.

Presentation outline

1. Introduction to Python Programming
2. **Introduction to Raspberry Pi and Robot HAT**
3. Introduction to Linux Command Line Interface (CLI)
4. Configuring and Integrating Robot Peripherals (Part 1)
   - Controlling LED (Warm color and WS2812)
   - Controlling DC motors
5. Advanced Programming Techniques
6. Configuring and Integrating Robot Peripherals (Part 2)
   - Multi-Threaded Control of Warm color LED
   - Controlling Servo motors
   - Interfacing with the Ultrasonic module
   - Interfacing with the Tracking module
   - Reading data from the MPU6050 module
   - Displaying Text on the OLED screen

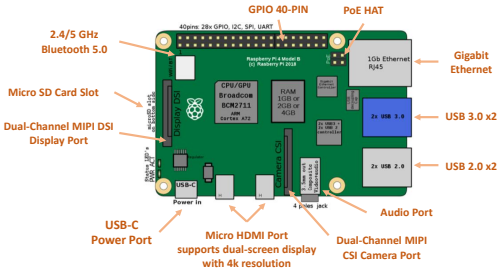
What is Raspberry Pi

- The Raspberry Pi is a small, affordable, single-board computer developed by the Raspberry Pi Foundation.
- The Raspberry Pi is capable of running a full operating system, and it's widely used for a variety of projects including robotics, home automation, media centers, and IoT (Internet of Things) devices.



Raspberry Pi 4 board

- In this course, we will use the Raspberry Pi 4



Raspberry Pi 4 board – Main modules

- **Broadcom BCM2711 System-on-Chip (SoC)**
  - **Processor**: Quad-core Cortex-A72 (ARM v8) 64-bit CPU, clocked at 1.5 GHz. This is the central processing unit (CPU) of the Raspberry Pi 4, providing the computational power.
  - **Graphics Processing Unit (GPU)**: Capable of handling 4K video output at 60fps.
- **Memory (RAM)**
  - The Raspberry Pi 4 comes with different RAM options: **2GB, 4GB, and 8GB of LPDDR4-3200 SDRAM**. This memory is used by the system to store data for active programs and processes.
- **Networking**
  - **Gigabit Ethernet Port**: Provides faster wired internet connection (up to 1 Gbps).
  - **Wireless Connectivity**: Dual-band 802.11ac Wi-Fi (2.4 GHz and 5.0 GHz) for wireless networking.
  - **Bluetooth 5.0**: For connecting peripherals such as keyboards, mice, or other devices.

Raspberry Pi 4 board – Main modules

- **USB Ports**
  - **2x USB 3.0 Ports**: For high-speed data transfer (up to 5 Gbps) to external devices.
  - **2x USB 2.0 Ports**: Standard USB ports for connecting peripherals such as keyboard, mouse, and USB drives.
- **Display Interface**
  - **2x Micro-HDMI Ports**: Support dual display output with resolutions up to 4K at 60fps. These are used to connect monitors or TVs.
  - **MIPI DSI Display Port**: For connecting compatible display panels, like the Raspberry Pi Touch Display.
- **Camera Interface**
  - **MIPI CSI Camera Port**: For connecting camera modules, such as the Raspberry Pi Camera Module, used for capturing images or video.

Raspberry Pi 4 board – Main modules

- **Audio/Video Output**
  - **3.5mm Audio and Composite Video Jack**: For audio output and older analog video output using RCA.
- **General-Purpose Input/Output (GPIO)**
  - **40 GPIO Pins**: These pins allow the Raspberry Pi to interface with various electronics, sensors, and other hardware. They support a variety of communication protocols like I2C, SPI, and UART.
- **Storage**
  - **Micro SD Card Slot**: Used for booting the operating system and storing files. Raspberry Pi OS (or another OS) is typically installed on the micro SD card.
- **Power**
  - **USB-C Power Port**: Requires a 5V, 3A power supply to provide power to the board and connected peripherals.

Operating System

- The Raspberry Pi supports a variety of operating systems, mainly based on Linux and Windows.
- Raspberry Pi OS (formerly Raspbian) is the official operating system for Raspberry Pi, based on **Debian Linux** and optimized for the Raspberry Pi's hardware.
- It provides a lightweight, user-friendly desktop environment and comes pre-installed with programming tools like Python.
- The OS supports a wide range of applications and provides full integration with Raspberry Pi hardware, including GPIO pins, camera modules, and other accessories.
- Raspberry Pi OS is available in both 32-bit and 64-bit versions, ensuring compatibility and better performance for Raspberry Pi 4.

Programming Language - Python

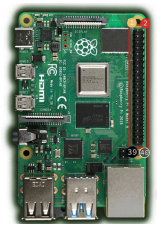
- **Python** is the most popular programming language for Raspberry Pi, and the "Pi" in Raspberry Pi is inspired by **Python**.
- It is a **high-level**, **interpreted**, and **object-oriented** programming language, known for its simplicity and readability.
- Python is **highly compatible** with the **Raspberry Pi's hardware**, and programs can be easily transferred across different Raspberry Pi models with minimal modification.
- **Official Documentation**: Visit python.org for extensive resources and tutorials.

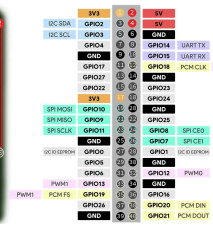
Key Features of Python

- **Ease of Use**: Python is easy to write, read, and maintain, making it perfect for beginners and advanced users alike.
- **Dynamic Typing**: Variable types are determined at runtime, allowing for flexible code development.
- **Object-Oriented**: Supports encapsulation, inheritance, and polymorphism, making it suitable for both small scripts and large applications.
- **Extensive Libraries**: Python offers a wide range of libraries for various applications, including robotics, web development, and IoT, all easily usable on Raspberry Pi.

Introduction to GPIO on Raspberry Pi

- **GPIO (General-Purpose Input/Output)** is a set of pins on the Raspberry Pi 4 that allows the board to interface with external electronics and devices.
  - These pins enable the Raspberry Pi to interact with a wide range of components like sensors, LEDs, motors, and more.





Introduction to GPIO on Raspberry Pi

- Three naming (coding) methods for Raspberry Pi pins
  - The **Wiring Pi number** is the pin number of the functional wiring (such as TXD, PWM0, etc.).
  - The **BCM number** is the Broadcom pin number, also known as GPIO.
  - The **physical number** is the number corresponding to the physical location of the pin on the Raspberry Pi motherboard (1 ~40).

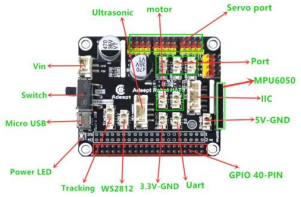
Physical Pins					
Function	BCM	pin#	pin#	BCM	Function
3.3 Volts		1	2		5 Volts
GPIO/SDA1 (I2C)	2	3	4		5 Volts
GPIO/SCL1 (I2C)	3	5	6		GND
GPIO/GCLK	4	7	8	14	TX UART/GPIO
GND		9	10	15	RX UART/GPIO
GPIO	17	11	12	18	GPIO
GPIO	27	13	14		GND
GPIO	22	15	16	23	GPIO
3.3 Volts		17	18	24	GPIO
MOSI (SPI)	10	19	20		GND
MISO(SPI)	9	21	22	25	GPIO
SCLK(SPI)	11	23	24	8	CE0_N (SPI)
GND		25	26	7	CE1_N (SPI)
RESERVED		27	28		RESERVED
GPIO	5	29	30		GND
GPIO	6	31	32	12	GPIO
GPIO	13	33	34		GND
GPIO	19	35	36	16	GPIO
GPIO	26	37	38	20	GPIO
GND		39	40	21	GPIO

Introduction to GPIO on Raspberry Pi

- **Digital Input/Output**: GPIO pins can be used as either inputs (to read signals like button presses) or outputs (to control devices like LEDs).
- **Communication Protocols**: Some GPIO pins are assigned for specific protocols such as:
  - I2C (Inter-Integrated Circuit)
  - SPI (Serial Peripheral Interface)
  - UART (Universal Asynchronous Receiver-Transmitter)
- **3.3V Logic**: GPIO pins on Raspberry Pi 4 operate at 3.3V logic levels, so caution is needed when interfacing with components to avoid damage.
- **Programming with Python**: You can control the GPIO pins using the `RPi.GPIO` or `gpiozero` libraries in Python. These libraries allow you to easily configure pins as input or output and to read/write values.

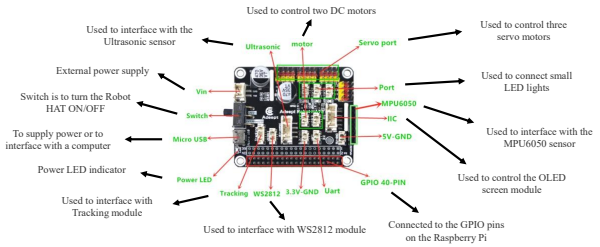
Robot HAT

- The **Adeept Robot HAT** board is a key component of the robot, providing multiple interfaces for connecting sensors and electronic hardware modules.
  - These interfaces enable the addition of various functionalities to the robot.
  - The system operates using a Raspberry Pi as the core controller.



Robot HAT

- The **Adeept Robot HAT** board is a key component of the robot, providing multiple interfaces for connecting sensors and electronic hardware modules.



Robot HAT – Precautions

- Do not use both the USB port and Vin on the Robot HAT to supply power at the same time.
- If you need to debug the program for a long time and don't want to remove the battery, you can set the switch on the Robot HAT to OFF, so that when you connect the Robot HAT with a USB cable, Robot HAT is powered by USB.
- If your robot reboots automatically after booting, or disconnects and reboots at the moment it starts to move after normal booting, it is likely that your power supply does not provide enough current.

Precautions when operating the robot

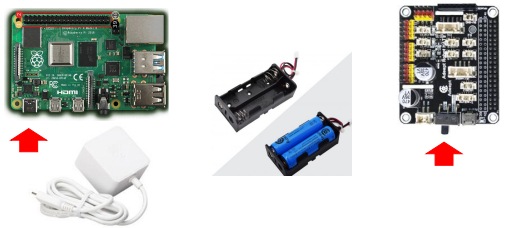
1. To prevent damage to the robot, **handle it with care** at all times.
2. Avoid applying excessive force to its components, ensure all connections are firmly secured, and operate it within the recommended speed and load limits.
3. Regularly inspect the robot for loose wires or parts, and **avoid sudden or abrupt movements** that could strain the motors or sensors.
4. Additionally and **most importantly**, never use both the USB port and the battery simultaneously to supply power to the robot.

Precautions when operating the robot

1. Switch off the Robot Hat
2. Remove the batteries
3. Connect the USB-C Power cable

OR

1. Disconnect the USB-C Power cable
2. Insert the batteries
3. Switch on the Robot Hat



Presentation outline

1. Introduction to Python Programming
2. Introduction to Raspberry Pi and Robot HAT
3. **Introduction to Linux Command Line Interface (CLI)**
4. Configuring and Integrating Robot Peripherals (Part 1)
   - Controlling LED (Warm color and WS2812)
   - Controlling DC motors
5. Advanced Programming Techniques
6. Configuring and Integrating Robot Peripherals (Part 2)
   - Multi-Threaded Control of Warm color LED
   - Controlling Servo motors
   - Interfacing with the Ultrasonic module
   - Interfacing with the Tracking module
   - Reading data from the MPU6050 module
   - Displaying Text on the OLED screen

Introduction to Linux Command Line Interface (CLI)

- **Definition:** The Linux Command Line Interface (CLI) is a text-based interface used to interact with the operating system.
  - Users input commands to perform tasks, which is crucial for Raspberry Pi, especially in robotics where automation and precision are essential.
- **Advantages:**
  - **Efficiency:** Direct access to system functions.
  - **Control:** Fine-grained control over software and hardware components.

Navigating the Filesystem commands

- `ls`: Lists files and directories.
  - Example: `ls -l` displays detailed information about each file.
  - Output example:
    ```
    drwxr-xr-x  2 user group 4096 Oct 10 14:20 scripts
    ```
- `cd`: Change directory.
  - Example: `cd /home/pi/robot` navigates to the "robot" directory.
- `pwd`: Shows the current working directory.
  - Example:
    ```
    /home/pi/robot
    ```


File and Directory Management commands

- `mkdir`: Create a new directory.
  - Example: `mkdir projects` creates a "projects" directory.
- `touch`: Create a new empty file.
  - Example: `touch robot.py` creates a Python file for your robot project.
- `mv`: Renaming a file in Linux is done using the `mv` command.
  - Example: `mv old_file.py new_file.py`
- `rm`: Remove files or directories.
  - Example:
    ```
    rm robot.py
    ```
  - Caution: Use `rm -r` for directories (e.g., `rm -r testdir`).

Viewing and Editing Files commands

- `cat`: Display the contents of a file.
  - Example: `cat robot.py` will print the script contents in the terminal.
- `nano` or `vim`: Text editors used within the terminal.
  - Example: `nano config.txt` opens the file for editing.
- `less` and `more`: View long files one screen at a time.
  - Example: `less longfile.txt` to scroll through a large file.

System Monitoring and Process Management commands

- `top`: Monitor system resource usage in real-time.
  - Example: Running `top` gives you a list of processes and their CPU usage.
- `ps`: List current running processes.
  - Example:
    `ps aux | grep python`
    This filters processes related to Python.
- `kill`: Terminate processes.
  - Example: `kill 1234` terminates the process with ID 1234.

Package Management with APT commands

- `apt update`: Refreshes the package list, ensuring you have the latest versions available.
- `apt install`: Installs packages.
  - Example:
    `sudo apt install python3`
    This installs Python 3 on your Raspberry Pi.
- `apt remove`: Uninstalls packages.
  - Example: `sudo apt remove python3` removes Python 3 from the system.

Basic Networking commands

- `ifconfig`: View and configure network settings.
  - Example: `ifconfig` displays current network interface configurations.
- `ping`: Test network connectivity.
  - Example:
    `ping 8.8.8.8`
    This checks connectivity to Google's DNS server.
- `scp`: Securely copy files between systems.
  - Example:
    `scp robot.py pi@192.168.0.10:/home/pi/`
    This transfers robot.py to a remote Raspberry Pi.

File Permissions and Ownership commands

- `chmod`: Modify file permissions.
  - Example:
    `chmod +x robot.py`
    This makes the robot.py script executable.
- `chown`: Change file owner.
  - Example:
    `sudo chown pi:pi robot.py`
    This transfers ownership of the file to the user pi.

Reboot and Shutdown commands

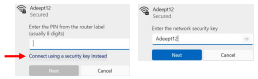
- Rebooting the System:
  `sudo reboot`
  - This command restarts your system immediately. Useful when making system-level changes or when the system needs to refresh.
- Shutting Down the System:
  `sudo shutdown now`
  - Shuts down the system immediately. You can also use the command:
  `sudo poweroff`
- Scheduled Shutdown:
  `Sudo shutdown +2`
  - Shuts down the system in 2 minutes.
- Canceling a Scheduled Shutdown
  `sudo shutdown -c`

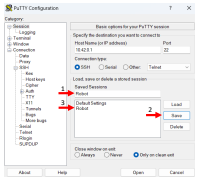
Presentation outline

1. Introduction to Python Programming
2. Introduction to Raspberry Pi and Robot HAT
3. Introduction to Linux Command Line Interface (CLI)
4. **Configuring and Integrating Robot Peripherals (Part 1)**
   - Controlling LED (Warm color and WS2812)
   - Controlling DC motors
5. Advanced Programming Techniques
6. Configuring and Integrating Robot Peripherals (Part 2)
   - Multi-Threaded Control of Warm color LED
   - Controlling Servo motors
   - Interfacing with the Ultrasonic module
   - Interfacing with the Tracking module
   - Reading data from the MPU6050 module
   - Displaying Text on the OLED screen

Access Raspberry Pi via WiFi using **PuTTY** (1/2)

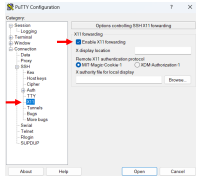
- Download and Install PuTTY:
  - Go to the official website: **putty.org**
  - Download and install PuTTY for your operating system (Windows)
- Connect to the Adeept Wi-Fi
  - On your computer, go to Wi-Fi settings.
  - Connect to the Wi-Fi network ***SSID : HotspotrobotXX***
  - Click on "*Connect using a security key instead*"
  - Enter the Wi-Fi password ***robot123*** when prompted.
- Open PuTTY:
  - Launch PuTTY on your computer.
  - In the Host Name (or IP Address) field, enter your Raspberry Pi's IP address "**10.42.0.1**".
  - Ensure the Port is set to **22** (for **SSH**).
  - Make sure the Connection type is set to **SSH**.





Access Raspberry Pi via WiFi using **PuTTY** (2/2)

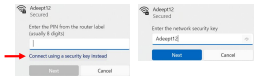
- Connect to the Raspberry Pi:
  - Enable **X11** forwarding, save your session for later use then click "Open" to initiate the connection.
  - If this is the first time connecting, PuTTY will display a security alert. Click "Accept" to proceed.
- Enter Login Credentials:
  - When prompted, enter the username "***robot00***" and press Enter.
  - Then, enter the password "***12345678***" and press Enter.
- You are now connected!
  - You now have SSH access to your Raspberry Pi through WiFi.

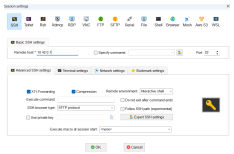




Access Raspberry Pi via WiFi using **MobaXterm** (1/2)

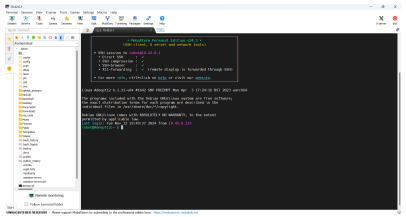
- Download and Install the free version of MobaXterm:
  - https://mobaxterm.mobatek.net/download.html
- Connect to the Adeept Wi-Fi
  - On your computer, go to Wi-Fi settings.
  - Connect to the Wi-Fi network ***HotspotrobotXX***.
  - Click on "*Connect using a security key instead*"
  - Enter the Wi-Fi password ***robot123*** when prompted.
- Open MobaXterm and Start a New SSH Session
  - Launch **MobaXterm**.
  - Click on the Session button at the top.
  - Select **SSH** as the session type (**Port 22**).
- Configure the SSH Session and connect to the Raspberry Pi:
  - In the SSH session settings, enter "**10.42.0.1"** as the Remote host (this is the IP address of the robot).
  - Make sure X11 is enabled in the Advanced SSH settings then click OK.





Access Raspberry Pi via WiFi using **MobaXterm** (2/2)

- Enter Login Credentials:
  - When prompted, enter the username "**robot00**" and press Enter.
  - Then, enter the password "**12345678**" and press Enter.
  - After successfully logging in, you'll have command-line access to your robot.



Geany Text Editor

- **Geany** is a powerful, stable and lightweight programmer's text **editor** that provides useful features.

  – For Linux: use **Putty** and launch **Geany**
  – For Windows: use **MobaXterm** and launch **Geany**

  - Linux command to launch Geany:
    `geany new_file.py &`

Connect **Mac** to Raspberry Pi via SSH over Wi-Fi

- Connect to the RPI Wi-Fi Network
  - Go to Wi-Fi settings on your Mac.
  - Connect to the network (e.g., Robot-WiFi).
  - Password: ***robot123***
- Open Terminal on Mac and enter SSH command:
  `ssh -X robot00@10.42.0.1 -p 22`
  - Explanation:
    - `ssh`: The command to start an SSH connection.
    - `-X`: Enables X11 forwarding, allowing GUI applications from the RPI to display on your Mac.
    - `robot00@10.42.0.1`: The username (robot) and IP address (10.42.0.1) of the Raspberry Pi.
    - `-p 22`: Specifies the port number (22 is the default SSH port).
- Enter Password "12345678" (when prompted).
- You're now connected and can control the RPI from Terminal.

Connecting to Your Raspberry Pi Robot via VNC Viewer on **Mac**

- Download and Install VNC Viewer
  - Go to ***realvnc.com*** and download **VNC Viewer** for macOS.
  - Open the downloaded file and drag VNC Viewer to your Applications folder to install it.
- Connect to the Robot's Wi-Fi Network
  - Go to Wi-Fi settings on your Mac.
  - Look for a Wi-Fi network named ***HotspotrobotXX***
  - Connect to this network using the password: ***robot123***
- Open VNC Viewer and Connect
  - Launch VNC Viewer from your Applications.
  - In the VNC Server field, enter the IP address of the Raspberry Pi robot "**10.42.0.1**".
  - Login: "**robot00**" and Password: "**12345678".**
  - Click Connect to access the robot's desktop.

Setting Up and Running a Python Script in Your Project Directory

- Create a directory for your work: `mkdir robot_team_xy`
- Navigate to the new directory: `cd robot_team_xy`
- Create a new Python file: `sudo touch test_robot_xy.py`
- Update file permissions: `sudo chmod 777 test_robot_xy.py`
- Open the file in the **Geany** editor: `geany test_robot_xy.py &`
- Write and save your Python code in the editor.
- Run your code: `sudo python3 test_robot_xy.py`

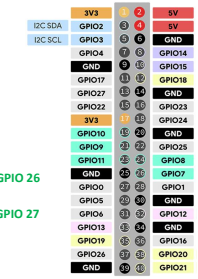
Configuring and Integrating Robot Peripherals



Raspberry Pi – Robot peripherals interfacing through GPIO

- **Front LEDs:**
  - Port 1 responsible of controlling **LED1**: **GPIO 5**
  - Port 2 responsible of controlling **LED2: GPIO 6**
  - Port 3 responsible of controlling **LED3: GPIO 13**
- **WS2812 3x RGB LED Module**: **GPIO 12**
- **DC motors:**
  - **Enable/Disable PIN** of the driver of **Motor A: GPIO 4**
  - **Controlling rotation of Motor A (forward/backward): GPIO 21** and **GPIO 26**
  - **Enable/Disable PIN** of the driver of **Motor B: GPIO 17**
  - **Controlling rotation of Motor B (forward/backward): GPIO 18** and **GPIO 27**





Presentation outline

1. Introduction to Python Programming
2. Introduction to Raspberry Pi and Robot HAT
3. Introduction to Linux Command Line Interface (CLI)
4. Configuring and Integrating Robot Peripherals (Part 1)
   - **Controlling LED (Warm color and WS2812)**
   - Controlling DC motors
5. Advanced Programming Techniques
6. Configuring and Integrating Robot Peripherals (Part 2)
   - Multi-Threaded Control of Warm color LED
   - Controlling Servo motors
   - Interfacing with the Ultrasonic module
   - Interfacing with the Tracking module
   - Reading data from the MPU6050 module
   - Displaying Text on the OLED screen

Controlling Warm color LEDs

- The warm color LED emits warm blue light with a power of 0.5W and operates at 5V or 12V. It has a 60° light-emitting angle. The red wire is the positive pole, and the black wire is the negative pole. The small LEDs connect to Port1, Port2, and Port3 on the Robot HAT driver board for use.

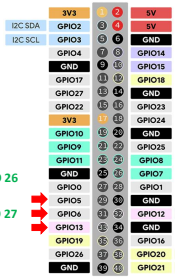


- The statement `import RPi.GPIO as GPIO` is used to import the `RPi.GPIO` library in Python.
  - `RPi.GPIO` Library: This is a Python library that provides an interface to control the GPIO (General Purpose Input/Output) pins on a Raspberry Pi.
    - These pins allow you to interact with external hardware like LEDs, sensors, and motors.
  - `as GPIO`: This part renames the library to GPIO for convenience, so you can use shorter and more readable code.
    - For example, instead of calling RPi.GPIO.setmode(), you can use GPIO.setmode().

Raspberry Pi – Robot peripherals interfacing through GPIO

- **Front LEDs:**
  - Port 1 responsible of controlling **LED1**: GPIO 5
  - Port 2 responsible of controlling **LED2:** GPIO 6
  - Port 3 responsible of controlling **LED3:** GPIO 13
- **WS2812 3x RGB LED Module**: GPIO 12
- **DC motors:**
  - **Enable/Disable PIN** of the driver of **Motor A:** GPIO 4
  - **Controlling rotation of Motor A (forward/backward):** GPIO 21 and GPIO 26
  - **Enable/Disable PIN** of the driver of **Motor B:** GPIO 17
  - **Controlling rotation of Motor B (forward/backward):** GPIO 18 and GPIO 27





Controlling Warm color LEDs

```
import RPi.GPIO as GPIO
import time

def switchSetup():                  # Python function designed to configure
                                    # specific GPIO pins on a Raspberry Pi for output.

        GPIO.setwarnings(False)     # Disables GPIO warnings, which can be helpful
                                    # to avoid repeated warnings if the pins are
                                    # reconfigured multiple times in the program.

        GPIO.setmode(GPIO.BCM)      # Sets the GPIO pin numbering mode to BCM
                                    # (Broadcom SOC channel numbers), which is
                                    # one of the ways to refer to the physical
                                    # pins on the Raspberry Pi.

        # PIN 5 connected to port 1 responsible of controlling LED1.
        GPIO.setup(5, GPIO.OUT)     # Configures pin 5 as output pins.
        # PIN 6 connected to port 2 responsible of controlling LED2.
        GPIO.setup(6, GPIO.OUT)     # Configures pin 6 as output pins.
        # PIN 13 connected to port 3 responsible of controlling LED3.
        GPIO.setup(13, GPIO.OUT)    # Configures pin 13 as output pins.
```


Controlling Warm color LEDs

- The switch function controls the high and low states of the specified interface. The parameter port represents ports 1 to 3. When status is 0, the corresponding interface light turns off, and when status is 1, the light turns on.

```
def switch(port, status):
    if port == 1:
        if status == 1:
            GPIO.output(5, GPIO.HIGH)
        elif status == 0:
            GPIO.output(5,GPIO.LOW)
        else:
            pass # Do nothing

    elif port == 2:
        if status == 1:
            GPIO.output(6, GPIO.HIGH)
        elif status == 0:
            GPIO.output(6,GPIO.LOW)
        else:
            pass # Do nothing
```

```
    elif port == 3:
            if status == 1:
                GPIO.output(13, GPIO.HIGH)
            elif status == 0:
                GPIO.output(13,GPIO.LOW)
            else:
                pass # Do nothing

        else:
            print('Wrong Command: Example--
switch(3, 1)->to switch on port3')
```


Controlling Warm color LEDs

```
def set_all_switch_off():           # The set_all_switch_off function sets all interfaces
                                    # to a low level, effectively turning off the lights
                                    # on all interfaces.
    switch(1,0)
    switch(2,0)
    switch(3,0)

if __name__ == "__main__":
    switchSetup()
    while 1:
        switch(1,1)
        switch(2,1)
        switch(3,1)
        print("Light on...")
        time.sleep(1)               # 1 second delay
        set_all_switch_off()
        print("Light off...")
        time.sleep(1)               # 1 second delay
```

To test the Warm color LEDs

Controlling WS2812 LED

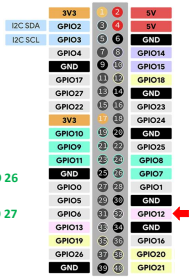
- The WS2812 RGB module is a low-power RGB LED. There are three WS2812 LEDs on each module.
  - The signal line needs to be connected to the "IN" port of WS2812 LED after being led from the Raspberry Pi.
  - When the next WS2812 LED needs to be connected, we connect a signal wire drawn from the "OUT" port of the previous WS2812 LED with the "IN" port of the next WS2812 LED.
  - Connecting the WS2812 LED to the WS2812 interface of Robot HAT is equivalent to connecting the signal line to the GPIO 12 of the Raspberry Pi.



Raspberry Pi – Robot peripherals interfacing through GPIO

- **Front LEDs:**
  - Port 1 responsible of controlling **LED1**: GPIO 5
  - Port 2 responsible of controlling **LED2:** GPIO 6
  - Port 3 responsible of controlling **LED3:** GPIO 13
- **WS2812 3x RGB LED Module**: GPIO 12
- **DC motors:**
  - **Enable/Disable PIN** of the driver of **Motor A:** GPIO 4
  - **Controlling rotation of Motor A (forward/backward):** GPIO 21 and GPIO 26
  - **Enable/Disable PIN** of the driver of **Motor B:** GPIO 17
  - **Controlling rotation of Motor B (forward/backward):** GPIO 18 and GPIO 27





Controlling WS2812 LED

```
import time
from rpi_ws281x import *

class LED:
    def __init__(self):
        self.LED_COUNT      = 3   # Set to the total number of LED lights on the robot
                                  # which can be more than the total number of LED lights
                                  # connected to the Raspberry Pi
        self.LED_PIN        = 12      # GPIO pin
        self.LED_FREQ_HZ    = 800000  # LED signal frequency in hertz (usually 800khz)
        self.LED_DMA        = 10      # DMA channel to use for generating signal
        self.LED_BRIGHTNESS = 255     # Set to 0 for darkest and 255 for brightest
        self.LED_INVERT     = False   # True to invert the signal
        self.LED_CHANNEL    = 0

        # Create NeoPixel object with appropriate configuration.
        self.strip = Adafruit_NeoPixel(self.LED_COUNT, self.LED_PIN, self.LED_FREQ_HZ,
                        self.LED_DMA, self.LED_INVERT, self.LED_BRIGHTNESS, self.LED_CHANNEL)

        # Intialize the library (must be called once before other functions).
        self.strip.begin()
```


Controlling WS2812 LED

```
    def colorWipe(self, R, G, B):
        # This function is used to change the color of the LED
        color = Color(R,G,B)
        for i in range(self.strip.numPixels()):
                # Only one LED light color can be set at a time, so a cycle is required
                self.strip.setPixelColor(i, color)
                self.strip.show() # After calling the show method, the color will really change

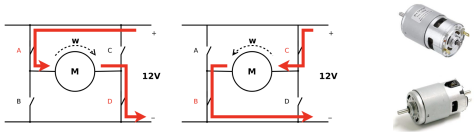

# This code will control all the WS2812 lights to switch among the three colors
# Press CTRL+C to exit the program.
if __name__ == '__main__':
        led = LED()
        try:
                while True:
                    led.colorWipe(255, 0, 0)  # All lights turn red
                    time.sleep(1)
                    led.colorWipe(0, 255, 0)  # All lights turn green
                    time.sleep(1)
                    led.colorWipe(0, 0, 255)  # All lights turn blue
                    time.sleep(1)
        except:
                led.colorWipe(0,0,0)  # Turn off all lights
```

Presentation outline

1. Introduction to Python Programming
2. Introduction to Raspberry Pi and Robot HAT
3. Introduction to Linux Command Line Interface (CLI)
4. Configuring and Integrating Robot Peripherals (Part 1)
   - Controlling LED (Warm color and WS2812)
   - **Controlling DC motors**
5. Advanced Programming Techniques
6. Configuring and Integrating Robot Peripherals (Part 2)
   - Multi-Threaded Control of Warm color LED
   - Controlling Servo motors
   - Interfacing with the Ultrasonic module
   - Interfacing with the Tracking module
   - Reading data from the MPU6050 module
   - Displaying Text on the OLED screen

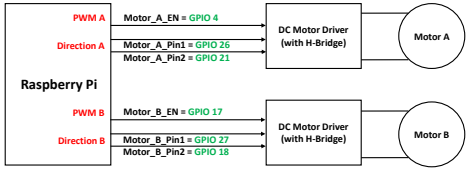
DC motors

- A DC motor is an electrical machine that converts electrical energy into mechanical energy. In a DC motor, the input electrical energy is the direct current which is transformed into the mechanical rotation.
  - **Direction Control**: Reversing the motor's direction is done by changing the voltage polarity, often using an H-bridge circuit.
  - **Speed Control**: Adjusting speed is achieved by varying the average voltage with pulse-width modulation (PWM), where the duty cycle of on/off pulses determines the effective voltage and speed.



DC motors

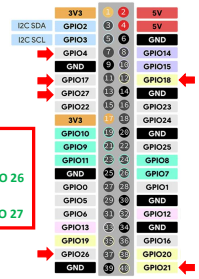
- The H-bridge is integrated into a circuit called a DC motor driver.
  - It acts as an interface between the Raspberry Pi and the motor.
  - The Raspberry Pi only needs to supply a PWM signal and the direction control.



Raspberry Pi – Robot peripherals interfacing through GPIO

- **Front LED:**
  - Port 1 responsible of controlling **LED1**: GPIO 5
  - Port 2 responsible of controlling **LED2:** GPIO 6
  - Port 3 responsible of controlling **LED3:** GPIO 13
- **WS2812 3x RGB LED Module**: GPIO 12
- **DC motors:**
  - **Enable/Disable PIN** of the driver of **Motor A:** GPIO 4
  - **Controlling rotation of Motor A (forward/backward):** GPIO 21 and GPIO 26
  - **Enable/Disable PIN** of the driver of **Motor B:** GPIO 17
  - **Controlling rotation of Motor B (forward/backward):** GPIO 18 and GPIO 27





DC motors – GPIO Functions

- `GPIO.output(channel, state)`: Sets the specified GPIO pin to HIGH (on) or LOW (off), allowing control of connected components like motors or LEDs.
  - `channel`: The GPIO pin number to set (e.g., 4, 17).
  - `state`: The desired output state (`GPIO.HIGH` or `GPIO.LOW`).
- `GPIO.setup(channel, mode, initial=None)`: Configures the specified GPIO pin as either an input or output, preparing it for interaction with external components.
  - `channel`: The GPIO pin number to configure (e.g., 4, 17).
  - `mode`: The configuration mode, such as `GPIO.OUT` (output) or `GPIO.IN` (input).
  - `initial (optional)`: The initial output state (`GPIO.HIGH` or `GPIO.LOW`) for output mode.
- `GPIO.setmode(mode)`: Sets the pin numbering convention for the script, ensuring consistent pin references throughout the code.
  - `mode`: The pin numbering system, either `GPIO.BCM` (Broadcom chip-specific numbering) or `GPIO.BOARD` (physical pin numbering).

DC motors – GPIO Functions

- `GPIO.PWM(channel, frequency)`: Initializes a PWM signal on the specified pin, allowing for precise control over motor speed or servo angles.
  - `channel`: The GPIO pin to use for PWM output (e.g., 4, 17).
  - `frequency`: The frequency of the PWM signal in Hz (e.g., 50 Hz for servo motors).
- `PWM.start(duty_cycle)`: Starts the PWM signal with the specified duty cycle to control connected devices.
  - `duty_cycle`: The initial duty cycle for the PWM signal, a value between 0 and 100 representing the proportion of the signal's on time.
- `PWM.ChangeDutyCycle(duty_cycle)`: Adjusts the duty cycle of an active PWM signal, changing the speed of a motor or position of a servo in real time.
  - `duty_cycle`: The new duty cycle to set, between 0 and 100.

DC motors

- When the DC Motor module is in use, it needs to be connected to the **motorA** or **motorB** interface on the Robot HAT drive board.
  - Built-in functions to control the DC motors:

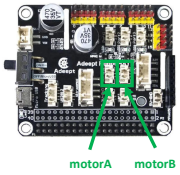
```
# Check the provided Python file:
# DCmotor_vS.py

setup()
# Motor initialization

motorStop()
# Motors stop

motor_right(status, direction, speed)
# Motor A positive and negative rotation
motor_left (status, direction, speed)
# Motor B positive and negative rotation

# status: 0 for inactive / 1 for active
# direction: forward/backward
# speed: from 0 (Off) to 100 (maximum speed)
```



DC motors – Tasks to be done – TSK01

- Develop functions:
  - To control the motor speed with a gradual acceleration.
  - To gradually slows down the motor until it stops.

Presentation outline

1. Introduction to Python Programming
2. Introduction to Raspberry Pi and Robot HAT
3. Introduction to Linux Command Line Interface (CLI)
4. Configuring and Integrating Robot Peripherals (Part 1)
   - Controlling LED (Warm color and WS2812)
   - Controlling DC motors
5. **Advanced Programming Techniques**
6. Configuring and Integrating Robot Peripherals (Part 2)
   - Multi-Threaded Control of Warm color LED
   - Controlling Servo motors
   - Interfacing with the Ultrasonic module
   - Interfacing with the Tracking module
   - Reading data from the MPU6050 module
   - Displaying Text on the OLED screen

Python – Classes

- Python is an **object-oriented programming (OOP)** language, and classes are the fundamental building blocks for creating objects in OOP.
- A **class** allows you to **define your own custom data types** by bundling **data** (***attributes***) and **methods** (***functions***) into a **single structure**.
  - **Syntax:**

```
class ClassName:
    # Class body
    # Define attributes and methods here
```

- **Why Use Classes?**
  - **Organization**: Classes help organize code into logical sections.
  - **Reusability**: Once a class is defined, you can create multiple objects (instances) based on it.
  - **Encapsulation**: Classes allow bundling of data and related behavior, hiding the internal implementation details.
  - **Abstraction**: They provide a clear structure to represent complex data.
  - **Inheritance**: Classes can be extended to create new classes, enabling code reuse.

Python – Classes

- A class is defined with the keyword ***class***, a ***class* name**, and a ***code* block**.
- Methods always take an extra argument, ***self***, and are called with the ***self* prefix** inside the class.
- Members (i.e. **variables**) in the class can be added at any time even outside of the class definition.
- Members are called internally with the ***self* prefix**.

```
class MyClass:
   # define a class member
   var1 = 1
   # and a class method
   def func1(self, x):
        return self.var1 + x

# make an object
mc = MyClass()
print(mc.var1)
# Output: 1

# Call the method
tmp = mc.func1(10)
# what's the value?
print(tmp)
# Output: 11

# Add a member to the class
mc.var2 = ['another', 'member']
print(mc.var2)
# Output: ['another', 'member']
```


Python – Classes

- Example of a Basic Class:

```
# Defining a simple class
class Person:
    # The constructor method to initialize attributes
    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age    # Instance attribute

    # Method to display information
    def greet(self):
        print(f"Hello, my name is {self.name} and I am {self.age} years old.")

# Creating an object (instance) of the class
person1 = Person("Alice", 30)
person1.greet()
# Output: Hello, my name is Alice and I am 30 years old.
```


Python – Classes

- **Attributes**:
  - **Instance Attributes**: Variables that belong to a specific object (instance) of the class.
    - They are defined using **`self`**.
  - **Class Attributes**: Variables that belong to the class itself and are shared by all instances.

```
class Car:
    # Class attribute
    wheels = 4

    def __init__(self, brand, model):
        # Instance attributes
        self.brand = brand
        self.model = model

car1 = Car("Toyota", "Corolla")
car2 = Car("Honda", "Civic")

print(car1.brand)      # Output: Toyota
print(Car.wheels)      # Output: 4 (class attribute)
```


Python – Classes

- **Methods**:
  - **Functions** defined inside a class are called **methods**.
  - The first parameter of a method is usually **`self`**, which refers to the current instance of the class.

```
class Calculator:
    def add(self, x, y):
        return x + y

    def subtract(self, x, y):
        return x - y

calc = Calculator()
print(calc.add(5, 3))         # Output: 8
print(calc.subtract(10, 4))   # Output: 6
```


Python – Classes

- Constructor (`__init__` Method):
  - The `__init__` method is a special method called a constructor. It is automatically called when a new object of the class is created.
  - It is used to initialize the object's attributes.

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        print(f"{self.name} says Woof!")

my_dog = Dog("Buddy", "Golden Retriever")
my_dog.bark()  # Output: Buddy says Woof!
```


Python – Classes

- **Inheritance**
  - Inheritance allows a class (*child* or *sub class*) to inherit attributes and methods from another class (*parent* or *super class*).
  - Sub classes get all of their parent's members and methods and can add their own.
  - It helps in reusing code and creating a relationship between classes.

```
# Base class (Parent class)
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound.")

# Derived class (Child class) inherits from Animal
class Dog(Animal):
    def speak(self):
        print(f"{self.name} barks.")

my_pet = Dog("Rex")
my_pet.speak()  # Output: Rex barks.
```


Python – Classes

- Example:

```
class Shape:
        # An empty base class
        pass
class Rectangle(Shape):
        # Inherits from Shape
        def __init__(self, length, width):
                self.length = length
                self.width = width

        def area(self):
                return self.length * self.width

class Square(Rectangle):
        def __init__(self, length):
                # Use the Rectangle initializer
                super().__init__(length, length)

rt = Rectangle(10.5, 4)
sq = Square(4)

rt.area()       # returns 42.0
sq.area()       # returns 16
```


Python – Classes

- **Encapsulation**
  - Encapsulation is the practice of **hiding** the internal details of an object and providing controlled access through methods.
  - In Python, you can make attributes private by prefixing them with double underscores (__).

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance  # Private attribute

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount("Alice", 1000)
account.deposit(500)
print(account.get_balance())       # Output: 1500
# print(account.__balance)         # Error: Attribute is private
```

Python – Classes

- In Python, `__name__` and `__main__` are special built-in variables that play a key role in controlling the execution of your code, especially when it comes to writing reusable modules and scripts.
  - `__name__` is a built-in variable in Python that represents the name of the current module. It is automatically set by Python when a module is run.
  - When a Python file is executed directly, the `__name__` variable is set to `__main__`.
  - `__main__` is the name of the scope in which the top-level code executes.
  - A file is considered the main program when it is run directly, and in that context, `__name__` is set to `__main__`.
- This mechanism allows Python to differentiate between a file being run as a standalone script or being imported as a module.

Python – Classes

```
# example.py
def greet():
    print("Hello from the greet function!")

# This code runs only if this file is executed directly
if __name__ == "__main__":
    print("This script is being run directly!")
    greet()
```

- If you run `example.py` directly:

```
python example.py

# Output:
# This script is being run directly!
# Hello from the greet function!
```

- `__name__` is set to `__main__`, so the if block runs, printing the message and calling `greet()`.

Python – Classes

- If you import `example.py` into another script:

```
# another_script.py
import example

# Output:
# (Nothing from the `if __name__ == "__main__":` block)
```

- `__name__` is set to `"example"` (the module name), so the if block does not run.

Python – Classes

- Organizing Code:
  - Imagine you have a file math_utils.py with utility functions:

```
# math_utils.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

# Test code
if __name__ == "__main__":
    print("Running test cases...")
    print("5 + 3 =", add(5, 3))          # Output: 8
    print("10 - 7 =", subtract(10, 7))   # Output: 3
```

  - The test code inside `if __name__ == "__main__":` will only execute if `math_utils.py` is run directly.

Python – Classes

- Organizing Code:
  - If `math_utils.py` is imported into another script:

```
# main_script.py
from math_utils import add

result = add(10, 20)
print("Result:", result)
```

  - The test code in `math_utils.py` will not run, making it a clean import.

Threading in Python

- Threading in Python allows you to run multiple operations concurrently in the same process.
  - It is useful for performing tasks simultaneously, such as handling I/O-bound operations or tasks that can be performed concurrently without needing separate processes.
  - Python provides a built-in threading module that makes it easy to work with threads.
- Basic Concepts of Threading
  - **Thread**: The smallest unit of a process that can be scheduled by the operating system.
  - **Main Thread**: The initial thread that runs when a Python script starts.
  - **Multithreading**: Running multiple threads at the same time to execute different tasks.

Threading in Python

- To use threading in Python, import the threading module and create threads using the Thread class.

```
import threading
import time

# Define three different functions for the threads
def print_numbers():
    for i in range(1, 6):
        print(f"Thread 1 - Number: {i}")
        time.sleep(1)  # Simulate a delay

def print_letters():
    for letter in 'ABCDE':
        print(f"Thread 2 - Letter: {letter}")
        time.sleep(1.5)  # Simulate a different delay

def print_symbols():
    for symbol in ['!', '@', '#', '$', '%']:
        print(f"Thread 3 - Symbol: {symbol}")
        time.sleep(2)  # Simulate another different delay

# Create three threads, each running a different function
thread1 = threading.Thread(target=print_numbers)
thread2 = threading.Thread(target=print_letters)
thread3 = threading.Thread(target=print_symbols)
```


Threading in Python

- To use threading in Python, import the threading module and create threads using the Thread class.

```
# Start the threads
thread1.start()
thread2.start()
thread3.start()

# Continue running other code while the threads run in the background
for i in range(10):
    print("Main thread is running while threads are executing.")
    time.sleep(1)

thread1.join()
thread2.join()
thread3.join()
# Wait for all threads to complete
print("All threads have completed.")
```


Mutlithreading – The Need for **Synchronization**

- Threads share memory, so if two threads modify the same variable at the same time, results can be unpredictable — a situation called a race condition.

```
import threading

counter = 0

def increment():
    global counter
    for _ in range(100000):
        counter += 1

threads = []
for _ in range(5):
    t = threading.Thread(target=increment)
    threads.append(t)
    t.start()

for t in threads:
    t.join()

print("Final counter:", counter)
```

You might expect `5 × 100000 = 500000`,



but the output is usually **less** because multiple threads read and write `counter` simultaneously, overwriting each other's updates.

Mutlithreading – Main Synchronization Techniques

- **Lock (Mutex):** A Lock ensures that only one thread can access a critical section at a time.

```
import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter
    for _ in range(100000):
        with lock:          # Acquire + release automatically
            counter += 1

threads = []
for _ in range(5):
    t = threading.Thread(target=increment)
    threads.append(t)
    t.start()

for t in threads:
    t.join()

print("Final counter:", counter)
```

Now you'll always get `500000`.
The `with lock:` ensures only one thread modifies `counter` at a time.

Mutlithreading – Main Synchronization Techniques

- **Rlock:** A **reentrant lock** allows a thread that already holds the lock to acquire it again. Useful in cases where functions that hold the lock call other functions that also try to acquire it.

```
import threading

lock = threading.RLock()

def outer():
    with lock:
        print("Outer lock acquired")
        inner()

def inner():
    with lock:
        print("Inner lock acquired")

threading.Thread(target=outer).start()
```

Without RLock, it would **deadlock**.

Mutlithreading – Main Synchronization Techniques

- **Semaphore:** A semaphore limits the number of threads that can access a resource simultaneously.

```
import threading
import time

semaphore = threading.Semaphore(3)  # Max 3 threads at once

def worker(n):
    with semaphore:
        print(f"Thread {n} entering")
        time.sleep(1)
        print(f"Thread {n} leaving")

for i in range(6):
    threading.Thread(target=worker, args=(i,)).start()
```

At most **3 threads** will run the "entering → leaving" block concurrently.

Mutlithreading – Main Synchronization Techniques

- **Event:** An Event allows one thread to signal one or more other threads. An **Event** is like a simple signal flag that can be **set** (True) or **cleared** (False). Threads (or processes) can **wait** for an event to be set before continuing. You can use multiple events to coordinate several stages in a workflow.

```
import threading
import time

event = threading.Event()

def waiter():
    print("Waiting for event to be set...")
    event.wait()  # Block until event is set
    print("Event received!")

def setter():
    time.sleep(3)
    print("Setting event.")
    event.set()

threading.Thread(target=waiter).start()
threading.Thread(target=setter).start()
```

The "waiter" thread blocks until `event.set()` is called.

Mutlithreading – Main Synchronization Techniques

- Using **Multiple Events**

```
import threading
import time

# Define three independent events

event_read = threading.Event()
event_process = threading.Event()
event_save = threading.Event()

def reader():
    print("[Reader] Waiting to start reading...")
    time.sleep(1)
    print("[Reader] Reading data...")
    time.sleep(2)
    print("[Reader] Done reading.")
    event_read.set()  # Signal that reading is complete

def processor():
    print("[Processor] Waiting for data to be read...")
    event_read.wait()  # Wait until reading is done
    print("[Processor] Processing data...")
    time.sleep(2)
    print("[Processor] Done processing.")
    event_process.set()  # Signal that processing is complete

def saver():
    print("[Saver] Waiting for processed data...")
    event_process.wait()  # Wait until processing is done
    print("[Saver] Saving data...")
    time.sleep(1)
    print("[Saver] Done saving.")
    event_save.set()  # Final event (optional)

# Start threads
threads = [
    threading.Thread(target=reader),
    threading.Thread(target=processor),
    threading.Thread(target=saver),
]

for t in threads:
    t.start()

# Wait for the final event to complete (logical synchronization)
event_save.wait()

# Then ensure all threads have truly ended (thread-level synchronization)
for t in threads:
    t.join()

print("All threads finished. Program exiting cleanly.")
```


Mutlithreading – Main Synchronization Techniques

- **Queue (Safe Data Sharing):** The easiest and safest way to share data between threads is using a **thread-safe queue** from the queue module. A **daemon** thread or process is a **background task** that automatically stops when the **main program ends**.

```
import threading
import queue
import time

q = queue.Queue()

def producer():
    for i in range(5):
        print(f"[Producer] Putting {i}")
        q.put(i)
        time.sleep(1)
    print("[Producer] Done producing.")

def consumer():
    while True:
        item = q.get()
        print(f"[Consumer] Got {item}")
        q.task_done()

# Consumer runs as daemon
consumer_thread = threading.Thread(target=consumer, daemon=True)
consumer_thread.start()

# Producer runs normally
producer_thread = threading.Thread(target=producer)
producer_thread.start()
producer_thread.join()

# Wait for queue to finish
q.join()
print("All items processed. Main thread exiting.")
```

`queue.Queue` handles all synchronization internally — no need for manual locks.

Interrupt Programming on Raspberry Pi

- In embedded systems, **interrupt programming** allows you to handle **events** or **changes** in **input signals immediately**, without continuous polling (checking in a loop).
  - This approach **saves CPU resources** and **improves efficiency**, making it ideal for handling input signals from sensors or other devices.
- On the Raspberry Pi, GPIO pins are used to connect to external components.
  - Using the RPi.GPIO library in Python, you can **set up interrupts on GPIO pins to respond to events** such as a **button press**, a **motion sensor trigger**, or a **change in signal level** from other sensors.

Interrupt Programming on Raspberry Pi – Example

- **Example: Using an Interrupt for a Motion Sensor (PIR) on the Raspberry Pi**
  - Let's say we have a **PIR (Passive Infrared)** motion sensor connected to GPIO pin 17, and we want to trigger an interrupt when motion is detected.
- To set up GPIO interrupts with Python on RPI:
  - Configure GPIO pins as input pins for the sensors.
  - Define a **callback function** to execute when an interrupt occurs.
  - Use `GPIO.add_event_detect()` to specify the interrupt and associate it with the **callback function**.

Interrupt Programming on Raspberry Pi – Example

```
import RPi.GPIO as GPIO
import time

# Configure the GPIO mode and setup
GPIO.setmode(GPIO.BCM)  # Use BCM pin numbering
PIR_PIN = 19  # GPIO pin connected to the PIR sensor

# Set up PIR_PIN as an input
GPIO.setup(PIR_PIN, GPIO.IN)

# Define a callback function to handle the interrupt
def motion_detected(channel):
    print("Motion detected!")

# Set up the interrupt with a callback on a RISING edge (motion detected)
GPIO.add_event_detect(PIR_PIN, GPIO.RISING, callback=motion_detected)

try:
    # Main loop to keep the program running
    print("Waiting for motion...")
    while True:
        time.sleep(1)  # Sleep to reduce CPU usage
except KeyboardInterrupt:
    print("Program interrupted by user")
```

Interrupt Programming on Raspberry Pi – Example

- Callback Function:
  - `motion_detected(channel)`: A function that prints "Motion detected!" whenever it is called by an interrupt event. This function will be triggered when the PIR sensor detects motion.
- Adding an Event Detect:
  - `GPIO.add_event_detect(PIR_PIN, GPIO.RISING, callback=motion_detected)`: Configures an interrupt on `PIR_PIN` that triggers on a RISING edge (when the signal goes from low to high, indicating motion) and calls the `motion_detected()` function.
- Other Edge Types:
  - `GPIO.RISING`: Trigger the interrupt on a low-to-high transition.
  - `GPIO.FALLING`: Trigger the interrupt on a high-to-low transition.
  - `GPIO.BOTH`: Trigger the interrupt on both transitions.

Tips for Writing Good Code in Python

- Use Meaningful Names for Variables, Functions, and Classes.
- Add comments to explain complex parts of the code or your logic.
- Use Built-in Functions and Libraries and organize your code with Functions and Classes:
  - Break down code into smaller, reusable functions to improve readability.
  - Use classes to encapsulate data and related functionality.
  - Keep functions short and focused on a single task (Single Responsibility Principle).
- Write Modular Code:
  - Split your code into smaller, logical files (modules) instead of writing everything in one file.
  - Use imports to bring in the functionality from other files when needed.
- Use try-except blocks to handle exceptions gracefully and prevent your program from crashing.
- Write unit tests to verify the functionality of your code.

Presentation outline

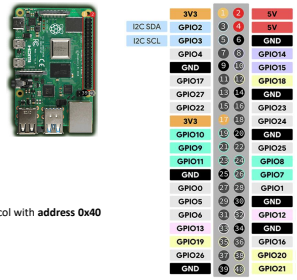
1. Introduction to Python Programming
2. Introduction to Raspberry Pi and Robot HAT
3. Introduction to Linux Command Line Interface (CLI)
4. Configuring and Integrating Robot Peripherals (Part 1)
   - Controlling LED (Warm color and WS2812)
   - Controlling DC motors
5. Advanced Programming Techniques
6. **Configuring and Integrating Robot Peripherals (Part 2)**
   - Multi-Threaded Control of Warm color LED
   - Controlling Servo motors
   - Interfacing with the Ultrasonic module
   - Interfacing with the Tracking module
   - Reading data from the MPU6050 module
   - Displaying Text on the OLED screen

Configuring and Integrating Robot Peripherals



Raspberry Pi – Robot peripherals interfacing through GPIO

- **Tracking module:**
  - **Left line IR sensor** is connected to **GPIO 19**
  - **Middle line IR sensor** is connected to **GPIO 16**
  - **Right line IR sensor** is connected to **GPIO 20**
- **Ultrasonic sensor:**
  - **Trig** pin of the ultrasonic sensor: **GPIO 11**
  - **Echo** pin of the ultrasonic sensor: **GPIO 8**
- **I2C COM** Protocol (**GPIO 2** and **GPIO 3**)
  - **Servo motors are controlled by the PCA9685 chip:** Interfacing to Raspberry Pi is done using I2C COM protocol with **address 0x40**
  - **MPU6050** uses I2C COM protocol with **address 0x68**
  - **OLED screen** uses I2C COM protocol with **address 0x3C**



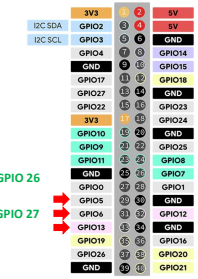
Presentation outline

1. Introduction to Python Programming
2. Introduction to Raspberry Pi and Robot HAT
3. Introduction to Linux Command Line Interface (CLI)
4. Configuring and Integrating Robot Peripherals (Part 1)
   - Controlling LED (Warm color and WS2812)
   - Controlling DC motors
5. Advanced Programming Techniques
6. Configuring and Integrating Robot Peripherals (Part 2)
   - **Multi-Threaded Control of Warm color LED**
   - Controlling Servo motors
   - Interfacing with the Ultrasonic module
   - Interfacing with the Tracking module
   - Reading data from the MPU6050 module
   - Displaying Text on the OLED screen

Raspberry Pi – Robot peripherals interfacing through GPIO

- **Front LED:**
  - Port 1 responsible of controlling **LED1**: **GPIO 5**
  - Port 2 responsible of controlling **LED2: GPIO 6**
  - Port 3 responsible of controlling **LED3: GPIO 13**
- **WS2812 3x RGB LED**: **GPIO 12**
- **DC motors:**
  - **Enable/Disable PIN** of the driver of **Motor A: GPIO 4**
  - **Controlling rotation of Motor A (forward/backward): GPIO 21** and **GPIO 26**
  - **Enable/Disable PIN** of the driver of **Motor B: GPIO 17**
  - **Controlling rotation of Motor B (forward/backward): GPIO 18** and **GPIO 27**





Multi-Threaded Control of Warm color LED

- The following code initializes GPIO pins and sets up two threads for controlling an LED and toggling its status.
- The main program turns the right and left LEDs on and off in intervals.
- The `LEDB()` thread blinks the LED based on `LEDB_status`, while `LEDB_statuschange()` toggles this status every 2 seconds.
- The `switch()` function controls the state of individual GPIO pins for the LEDs.
- The **try-except block** ensures that threads are safely stopped when the program is interrupted.

Multi-Threaded Control of Warm color LED

```
import RPi.GPIO as GPIO                 # Check the provided Python file:
import threading                        # FLED_vS.py
import time

LEDB_status = 1  # Initial status of the LED
stop_thread = False  # Flag to control when to stop the threads

def switchSetup():
    # Set up GPIO mode and configure pins as outputs
    GPIO.setwarnings(False)
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(5, GPIO.OUT)             # Pin 5 for Head LED
    GPIO.setup(6, GPIO.OUT)             # Pin 6 for Right LED
    GPIO.setup(13, GPIO.OUT)            # Pin 5 for Left LED

def LEDB_statuschange():
    # Toggle LEDB_status every 2 seconds
    global LEDB_status
    while not stop_thread:
        LEDB_status = not LEDB_status
        time.sleep(2)
```

Multi-Threaded Control of Warm color LED

```
def LEDB():
    # Blink the LED connected to pin 5 based on LEDB_status
    global LEDB_status
    while not stop_thread:
        if LEDB_status == 1:
            GPIO.output(5, GPIO.HIGH)  # Turn on Head LED
            time.sleep(0.1)
        GPIO.output(5, GPIO.LOW)  # Turn off Head LED
        time.sleep(0.1)

def switch(port, status):
    # Control the output of pins 6 and 13 based on the port and status
    if port == 0:  # Right switch
        if status == 1:
            GPIO.output(6, GPIO.HIGH)  # Turn on right LED
        elif status == 0:
            GPIO.output(6, GPIO.LOW)  # Turn off right LED
    elif port == 1:  # Left switch
        if status == 1:
            GPIO.output(13, GPIO.HIGH)  # Turn on left LED
        elif status == 0:
            GPIO.output(13, GPIO.LOW)  # Turn off left LED
    else:
        print('Wrong Command.')  # Handle invalid port numbers
```

```
def set_all_switch_off():
    # Turn off all switches (both LEDs)
    switch(0, 0)
    switch(1, 0)
```


Multi-Threaded Control of Warm color LED

```
if __name__ == "__main__":
    switchSetup()  # Set up GPIO configuration
    # Create and start threads for LED control and status change
    thread1 = threading.Thread(target=LEDB)
    thread2 = threading.Thread(target=LEDB_statuschange)
    thread1.start()
    thread2.start()
    try:
        # Main loop to turn LEDs on and off with 0.5-second intervals
        while True:
            switch(0, 1)  # Turn on right LED
            switch(1, 1)  # Turn on left LED
            print("Light on...")
            time.sleep(0.5)  # Wait 0.5 seconds
            set_all_switch_off()  # Turn off both LEDs
            print("Light off...")
            time.sleep(0.5)  # Wait 0.5 seconds
    except:
        # Stop threads safely on program interruption
        stop_thread = True
        print('Threads are requested to stop safely - please wait!')
        time.sleep(2)  # Wait for threads to finish safely
        print('Threads have been stopped.')
```


Controlling LED – Tasks to be done – TSK02

- Demonstrate LED control using **threads** for different delay and blinking configurations.
  - Create functions for each LED with a unique blinking pattern.
  - Use Python's threading module to run each LED function in separate threads with varying delays.
  - Start and stop threads to show concurrent blinking patterns.

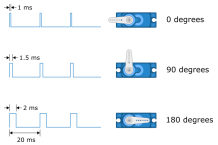
Presentation outline

1. Introduction to Python Programming
2. Introduction to Raspberry Pi and Robot HAT
3. Introduction to Linux Command Line Interface (CLI)
4. Configuring and Integrating Robot Peripherals (Part 1)
   - Controlling LED (Warm color and WS2812)
   - Controlling DC motors
5. Advanced Programming Techniques
6. Configuring and Integrating Robot Peripherals (Part 2)
   - Multi-Threaded Control of Warm color LED
   - **Controlling Servo motors**
   - Interfacing with the Ultrasonic module
   - Interfacing with the Tracking module
   - Reading data from the MPU6050 module
   - Displaying Text on the OLED screen

Servo motors

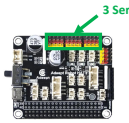
- Servo motors are precision-controlled motors widely used in robotics for tasks that demand accurate positioning. Unlike standard DC motors, servo motors can rotate to specific angles, making them ideal for applications such as steering, robotic arms, and camera stabilization.
- Servo motors are controlled by pulse-width modulation (PWM), where the angle is determined by the length of the control pulse:
  - **Pulse Width**: Typically, a 1ms pulse moves the servo to 0°, 1.5ms to 90°, and 2ms to 180°, though these values can vary by servo model.
  - **PWM Signal Frequency**: Standard servos operate at a 50Hz frequency (20ms cycle), with each pulse determining the motor position.





Servo motors

- The included servo motor can rotate between 0° and 180°.
  - This 180° servo is popular in robotics as it allows control over the rotation angle of various mechanisms via PWM signals.
- On the Raspberry Pi's Robot HAT driver board, a PCA9685 chip is dedicated to controlling servos.
  - The Raspberry Pi communicates with the PCA9685 using the I2C protocol, sending pulse signals from the microcontroller to guide the servo's movement. These pulses indicate the target position for the servo.

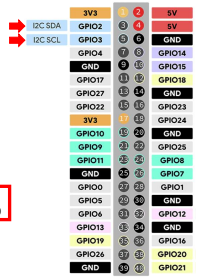




Raspberry Pi – Robot peripherals interfacing through GPIO

- **Tracking module:**
  - **Left line IR sensor** is connected to **GPIO 19**
  - **Middle line IR sensor** is connected to **GPIO 16**
  - **Right line IR sensor** is connected to **GPIO 20**
- **Ultrasonic sensor:**
  - **Trig** pin of the ultrasonic sensor: **GPIO 11**
  - **Echo** pin of the ultrasonic sensor: **GPIO 8**
- **I2C COM** Protocol (**GPIO 2** and **GPIO 3**)
  - **Servo motors are controlled by the PCA9685 chip:**
    Interfacing to Raspberry Pi is done using I2C COM protocol with **address 0x40**
  - **MPU6050** uses I2C COM protocol with **address 0x68**
  - **OLED screen** uses I2C COM protocol with **address 0x3C**





Servo motors

- Functions of `Adafruit_PCA9685` module – `PCA9685` Class

  – `set_pwm_freq(freq)` is used to set the PWM frequency.
  - This setting depends on the model of the servo. The included servo motor needs to be controlled by a 50 Hz PWM signal.

  – `pwm.set_pwm(PN, DV, PWM_DC)` is used to control the rotation of a servo to a certain position.
  - `PN` is the port number of the servo motor.
  - `DV` is the deviation value for controlling the rotation of the steering gear, which is not used in our program.
  - `PWM_DC` is the PWM duty cycle range of the servo we use is about 100 to 560, which corresponds to a rotation range of about 0° to 180°.

Servo motors

- Functions of `RPIservo_vS` module – `ServoCtrl` Class

```
# Check the provided Python file:
# RPIservo_vS.py


def __init__(self):
    # Initialize servo control parameters
    self.sc_direction = [1, 1, 1]          # Direction multipliers for each servo
    self.initPos = [init_pwm0, init_pwm1, init_pwm2]  # Initial PWM positions for each servo
    self.nowPos  = [300, 300, 300]         # Current positions of each servo
    self.lastPos = [300, 300, 300]         # Last recorded positions of each servo
    self.maxPos  = [360, 360, 360]         # Maximum PWM position limits for each servo
    self.minPos  = [240, 240, 240]         # Minimum PWM position limits for each servo

    self.ctrlRangeMax = 360                # Maximum control range for PWM output
    self.ctrlRangeMin = 240                # Minimum control range for PWM output
    self.angleRange = 180                  # Total angle range for the servo (degrees)
```

Servo motors

- Functions of `RPIservo_vS` module – `ServoCtrl` Class

```
def moveInit(self):
    # Initialize each servo position to its starting position
    for i in range(0, 3):
        pwm.set_pwm(i, 0, self.initPos[i])   # Set initial PWM for each servo channel
        self.lastPos[i] = self.initPos[i]    # Set last position to initial position
        self.nowPos[i] = self.initPos[i]     # Set current position to initial position

def pwmGenOut(self, angleInput):
    # Generate the PWM output for a given angle
    # Scales the input angle within the specified control range
    return int(round(((self.ctrlRangeMax - self.ctrlRangeMin) / self.angleRange * angleInput), 0))
```


Servo motors

- Functions of `RPIservo_vS` module – `ServoCtrl` Class

```
def moveAngle(self, ID, angleInput):
    # Calculate the new position for the servo based on the input angle.
    self.nowPos[ID] = int(self.initPos[ID] + self.sc_direction[ID] * self.pwmGenOut(angleInput))

    # Ensure the calculated position does not exceed the maximum allowed position.
    if self.nowPos[ID] > self.maxPos[ID]:
        self.nowPos[ID] = self.maxPos[ID]
    # Ensure the calculated position does not go below the minimum allowed position.
    elif self.nowPos[ID] < self.minPos[ID]:
        self.nowPos[ID] = self.minPos[ID]

    # Update the last known position of the servo.
    self.lastPos[ID] = self.nowPos[ID]

    # Send the calculated position to the servo using PWM.
    pwm.set_pwm(ID, 0, self.nowPos[ID])
```


Servo motors – Multi-Threaded Based Control

```
import time
import RPi.GPIO as GPIO
import threading
import Adafruit_PCA9685
import DCmotor_vS as move
import RPIservo_vS as RPIservo

left_forward = 1
left_backward = 0

right_forward = 1
right_backward = 0

move.setup()
scGear = RPIservo.ServoCtrl()
scGear.moveInit()

pwm = Adafruit_PCA9685.PCA9685()
pwm.set_pwm_freq(50)
```

```
def ctrl_motors():
        move.motor_left(1, left_forward, 60)
        move.motor_right(1, right_forward, 60)
        time.sleep(1)
        move.motor_left(1, left_forward, 30)
        move.motor_right(1, right_forward, 30)
        time.sleep(1)
        move.motor_left(1, left_backward, 60)
        move.motor_right(1, right_backward, 60)
        time.sleep(1)
        move.motor_left(1, left_backward, 30)
        move.motor_right(1, right_backward, 30)
        time.sleep(1)

def ctrl_wheels_pwm():
        scGear.moveAngle(0,-40)
        time.sleep(1)
        scGear.moveAngle(0,0)
        time.sleep(2)
        scGear.moveAngle(0,40)
        time.sleep(1)
```


Servo motors – Multi-Threaded Based Control

```
def ctrl_head_LR_pwm():
        scGear.moveAngle(1,-40)
        time.sleep(1)
        scGear.moveAngle(1,0)
        time.sleep(2)
        scGear.moveAngle(1,40)
        time.sleep(1)

def ctrl_head_UD_pwm():
        scGear.moveAngle(2,-45)
        time.sleep(1)
        scGear.moveAngle(2,0)
        time.sleep(1)
        scGear.moveAngle(2,45)
        time.sleep(1)
        scGear.moveAngle(2,0)
        time.sleep(1)

thread1 = threading.Thread(target = ctrl_motors)
thread2 = threading.Thread(target = ctrl_wheels_pwm)
thread3 = threading.Thread(target = ctrl_head_LR_pwm)
thread4 = threading.Thread(target = ctrl_head_UD_pwm)
```

```
thread1.start()
thread2.start()
thread3.start()
thread4.start()

thread1.join()
thread2.join()
thread3.join()
thread4.join()

print("Threads have been stopped.")
```


Servo motors – Tasks to be done – TSK03

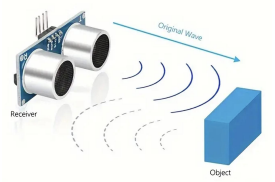
- Part A: Testing the Servo Motor Controlling the Robot's Direction
  - Develop a function to control the servo motor for smooth left and right turns of the robot's front wheels and test to ensure smooth directional movement.
- Part B: Changing Head Direction (Ultrasonic Sensor and Camera)
  - Create functions to rotate the head (holding the ultrasonic sensor and camera) in different directions using the two additional servos to control the head's up/down and left/right movement, test to ensure for accurate positional adjustments.
- Part C: Including DC Motors for Enhanced Rotation of the Robot
  - Combine front wheel steering with back DC motors rotating in opposite directions.
  - Test various rotation configurations, observing if the robot turns more efficiently.

Presentation outline

1. Introduction to Python Programming
2. Introduction to Raspberry Pi and Robot HAT
3. Introduction to Linux Command Line Interface (CLI)
4. Configuring and Integrating Robot Peripherals (Part 1)
   - Controlling LED (Warm color and WS2812)
   - Controlling DC motors
5. Advanced Programming Techniques
6. Configuring and Integrating Robot Peripherals (Part 2)
   - Multi-Threaded Control of Warm color LED
   - Controlling Servo motors
   - **Interfacing with the Ultrasonic module**
   - Interfacing with the Tracking module
   - Reading data from the MPU6050 module
   - Displaying Text on the OLED screen

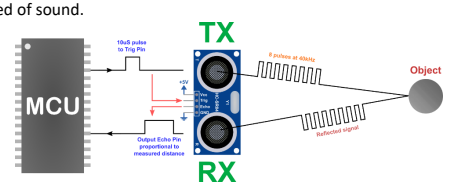
Digital Ultrasonic Sensor – HC-SR04

- The HC-SR04 ultrasonic sensor is a widely used electronic device for non-contact distance measurement using ultrasonic sound waves.
  - It sends a burst of ultrasonic waves at a specified frequency, typically 40 kHz.
  - These waves travel through the air and reflect back to the sensor when they hit an object in the beam's path.



Digital Ultrasonic Sensor – HC-SR04

- The elapsed time between emitting the ultrasonic signal and receiving its reflection is directly related to the distance between the sensor and the object.
  - By using the speed of sound in air (approximately 340 m/s at 20°C), the distance can be calculated by multiplying the round-trip time by half the speed of sound.



Digital Ultrasonic Sensor – HC-SR04

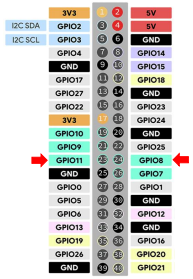
- **Measurement Range**: The typical range for the HC-SR04 sensor is 2 cm to 400 cm.
- **Accuracy**: Accuracy can reach 3 mm but it varies with environmental conditions and the quality of signal reflection, but it's generally suitable for many applications.
- **Detection Angle**: The HC-SR04 emits sound within a detection angle of about 15 degrees, allowing it to sense objects in a 30-degree cone.
- **Response Time**: The sensor responds quickly, usually within milliseconds.
- **Power Supply**: It operates on a 5V DC power source.

Raspberry Pi – Robot peripherals interfacing through GPIO

- **Tracking module:**
  - **Left line IR sensor** is connected to **GPIO 19**
  - **Middle line IR sensor** is connected to **GPIO 16**
  - **Right line IR sensor** is connected to **GPIO 20**

- **Ultrasonic sensor:**
  - **Trig** pin of the ultrasonic sensor: **GPIO 11**
  - **Echo** pin of the ultrasonic sensor: **GPIO 8**





- **I2C COM** Protocol (**GPIO 2** and **GPIO 3**)
  - **Servo motors are controlled by the PCA9685 chip:**
    Interfacing to Raspberry Pi is done using I2C COM protocol with **address 0x40**
  - **MPU6050** uses I2C COM protocol with **address 0x68**
  - **OLED screen** uses I2C COM protocol with **address 0x3C**

Digital Ultrasonic Sensor – HC-SR04

```
import RPi.GPIO as GPIO
# Import the RPi.GPIO library for controlling the GPIO pins on the Raspberry Pi
import time
# Import the time module to add delays and measure time intervals

# Define the GPIO pins for the ultrasonic sensor's trigger and echo
Trig = 11  # Pin number for the trigger pin (configured as an output)
Echo = 8   # Pin number for the echo pin (configured as an input)

def checkdist():  # Function to measure the distance using the ultrasonic sensor
    # Set up the GPIO mode to use Broadcom (BCM) numbering
    GPIO.setmode(GPIO.BCM)
    # Configure the Trig pin as an output and set its initial state to LOW
    GPIO.setup(Trig, GPIO.OUT, initial=GPIO.LOW)
    # Configure the Echo pin as an input
    GPIO.setup(Echo, GPIO.IN)
```


Digital Ultrasonic Sensor – HC-SR04

```
    # Send a short HIGH pulse to the Trig pin to trigger the ultrasonic burst
    GPIO.output(Trig, GPIO.HIGH)
    time.sleep(0.000015)  # Wait for 15 microseconds (the required pulse width)

    # Set the Trig pin to LOW to stop the pulse
    GPIO.output(Trig, GPIO.LOW)

    # Wait until the Echo pin goes HIGH, indicating that the sound wave has been emitted
    while not GPIO.input(Echo):
        pass  # Wait in a loop until the Echo pin goes HIGH

    # Record the start time when the Echo pin goes HIGH
    t1 = time.time()
```


Digital Ultrasonic Sensor – HC-SR04

```
    # Wait until the Echo pin goes LOW, indicating that the sound wave has returned
    while GPIO.input(Echo):
        pass  # Wait in a loop until the Echo pin goes LOW

    # Record the end time when the Echo pin goes LOW
    t2 = time.time()

    # Calculate the distance based on the time difference and the speed of sound (343 m/s)
    # The division by 2 accounts for the round-trip time of the sound wave
    return round((t2 - t1) * 34300 / 2, 2)  # Return the distance rounded to two decimal places
```


Digital Ultrasonic Sensor – HC-SR04

- Main problem: **Busy-wait loops**

```
while not GPIO.input(Echo):
    pass

while GPIO.input(Echo):
    pass
```

- These two loops are **blocking** and **CPU-intensive**:
  - The Raspberry Pi's CPU continuously polls the GPIO pin state.
  - It prevents multitasking or parallel operations.
  - Timing accuracy may degrade under system load.

- **Better Solution using Interrupts**, you can:
  - Register callback functions that trigger when Echo changes state.
  - Let the CPU handle other tasks (or sleep) in the meantime.
  - Achieve more accurate and responsive timing.

Digital Ultrasonic Sensor – HC-SR04

- **Using Interrupts**

```
import RPi.GPIO as GPIO
import time

# Pin Configuration (BCM mode)
TRIG = 11   # GPIO pin for Trigger
ECHO = 8    # GPIO pin for Echo

# Global variables for timing
start_time = 0
end_time = 0
distance_ready = False

# Callback function triggered by Echo pin edges
def echo_callback(channel):
    global start_time, end_time, distance_ready
    if GPIO.input(ECHO):  # Rising edge: signal sent
        start_time = time.time()
    else:                 # Falling edge: signal received back
        end_time = time.time()
        distance_ready = True

# Setup Function
def setup():
    GPIO.setmode(GPIO.BCM)
    GPIO.setwarnings(False)

    GPIO.setup(TRIG, GPIO.OUT)
    GPIO.setup(ECHO, GPIO.IN)

    # Ensure the trigger pin starts LOW
    GPIO.output(TRIG, GPIO.LOW)
    time.sleep(0.1)

    # Attach interrupt on both rising and falling edges
    GPIO.add_event_detect(ECHO, GPIO.BOTH, callback=echo_callback)
```


Digital Ultrasonic Sensor – HC-SR04

- **Using Interrupts**

```
# Function to measure one distance
def measure_distance():
    global distance_ready

    # Send a 15 us trigger pulse
    GPIO.output(TRIG, GPIO.HIGH)
    time.sleep(0.000015)
    GPIO.output(TRIG, GPIO.LOW)

    # Wait for measurement result or timeout
    timeout = time.time() + 0.05  # 50 ms
    while not distance_ready and time.time() < timeout:
        time.sleep(0.001)

    if distance_ready:
        distance_ready = False
        duration = end_time - start_time
        # Calculate distance (speed of sound = 343 m/s)
        distance_cm = (duration * 34300) / 2
        return round(distance_cm, 2)
    else:
        return None  # Timeout or no echo detected
```

```
# Main program
def main():
    setup()
    print("Ultrasonic distance measurement started (Ctrl+C to stop)")
    try:
        while True:
            dist = measure_distance()
            if dist is not None:
                print(f"Distance: {dist} cm")
            else:
                print("No echo detected")
            time.sleep(0.5)

    except KeyboardInterrupt:
        print("\nMeasurement stopped by user.")

    finally:
        GPIO.cleanup()
        print("GPIO cleaned up. Exiting.")

# Entry point
if __name__ == "__main__":
    main()
```


Digital Ultrasonic Sensor – Tasks to be done – TSK04

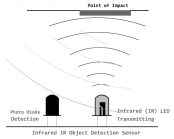
- Develop functions to operate the ultrasonic sensor with head servo to detect obstacles in specific directions:
  - Sweep the servo from left to right.
  - Use the ultrasonic sensor to measure and display distances at each position.
  - Print direction and distance on the console.
  - Generate an alert or generate commands to stop the robot when the detected obstacle is too close.

Presentation outline

1. Introduction to Python Programming
2. Introduction to Raspberry Pi and Robot HAT
3. Introduction to Linux Command Line Interface (CLI)
4. Configuring and Integrating Robot Peripherals (Part 1)
   - Controlling LED (Warm color and WS2812)
   - Controlling DC motors
5. Advanced Programming Techniques
6. Configuring and Integrating Robot Peripherals (Part 2)
   - Multi-Threaded Control of Warm color LED
   - Controlling Servo motors
   - Interfacing with the Ultrasonic module
   - **Interfacing with the Tracking module**
   - Reading data from the MPU6050 module
   - Displaying Text on the OLED screen

Digital IR (Infra-Red) Proximity Sensor

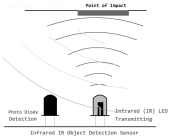
- The IR proximity sensor is an electronic device designed to detect the presence of nearby objects using infrared light beams.
- This type of sensor is widely used in automation systems, robotics, consumer electronics, and security devices.
- The detection range of an IR proximity sensor can vary from a few centimeters to several meters, depending on the model.

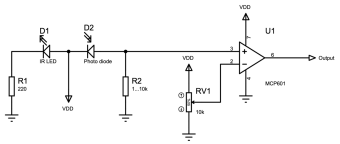




Digital IR (Infra-Red) Proximity Sensor

- The sensor emits an infrared light beam that is invisible to the human eye.
  - When this infrared beam encounters an object, it reflects back toward the sensor.
  - IR proximity sensors often have a fast response time, making them suitable for applications requiring quick object detection.



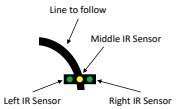




Tracking module

- A tracking module based on 3 IR sensors is used in line follower robots to detect and follow a black line on a light-colored surface.
  - Each IR sensor consists of an IR transmitter and an IR receiver.
  - The module operates by emitting infrared light and detecting the reflection.

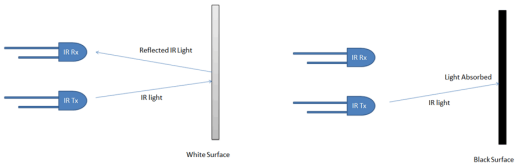




- How It Works:
  - IR Light Emission: The IR sensors emit infrared light towards the surface.
  - Sensor Placement: The 3 sensors are positioned side by side on the front of the robot to scan the path.
    - The middle sensor helps keep the robot on the line.
    - The left and right sensors detect deviations from the path, enabling the robot to adjust its direction to stay on the line.

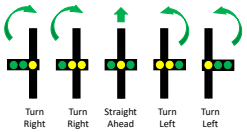
Tracking module

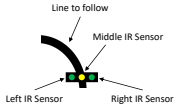
- How It Works:
  - Reflection Detection: The receiver detects the amount of reflected IR light.
    - Light-colored surfaces reflect more IR light, resulting in a strong signal.
    - Dark-colored surfaces (e.g., black lines) absorb more IR light, resulting in a weaker signal.



Tracking module

- Operation in a Line Follower Robot:





  - When the robot moves:
    - If the middle sensor is over the black line (low reflection), it indicates the robot is on track.
    - If the line shifts to the left or right, one of the side sensors will detect it (low reflection), prompting the robot to adjust its direction to re-center itself on the line.
    - The robot continuously adjusts based on real-time feedback from the sensors to follow the path accurately.

Raspberry Pi – Robot peripherals interfacing through GPIO

- **Tracking module:**
  - **Left line IR sensor** is connected to **GPIO 19**
  - **Middle line IR sensor** is connected to **GPIO 16**
  - **Right line IR sensor** is connected to **GPIO 20**
- **Ultrasonic sensor:**
  - **Trig** pin of the ultrasonic sensor: **GPIO 11**
  - **Echo** pin of the ultrasonic sensor: **GPIO 8**
- **I2C COM** Protocol (**GPIO 2** and **GPIO 3**)
  - **Servo motors are controlled by the PCA9685 chip:** Interfacing to Raspberry Pi is done using I2C COM protocol with **address 0x40**
  - **MPU6050** uses I2C COM protocol with **address 0x68**
  - **OLED screen** uses I2C COM protocol with **address 0x3C**





Tracking module

```
import time                                          # Check the provided Python file:
import RPi.GPIO as GPIO                              # Tracking_v8.py

# Define GPIO pins connected to the line sensors
line_pin_left = 19     # Left line sensor connected to GPIO 19
line_pin_middle = 16   # Middle line sensor connected to GPIO 16
line_pin_right = 20    # Right line sensor connected to GPIO 20

def setup():
    # Set up GPIO settings for the line sensors
    GPIO.setwarnings(False)          # Disable GPIO warnings
    GPIO.setmode(GPIO.BCM)           # Set GPIO mode to BCM (Broadcom pin numbering)

    # Configure each line sensor pin as an input
    GPIO.setup(line_pin_right, GPIO.IN)     # Right sensor as input
    GPIO.setup(line_pin_middle, GPIO.IN)    # Middle sensor as input
    GPIO.setup(line_pin_left, GPIO.IN)      # Left sensor as input

def run():
    # Read the current state of each line sensor
    status_right = GPIO.input(line_pin_right)   # Read the right sensor's status
    status_middle = GPIO.input(line_pin_middle) # Read the middle sensor's status
    status_left = GPIO.input(line_pin_left)     # Read the left sensor's status

    # Print the status of each sensor in a formatted string
    print('LS: %d   MS: %d   RS: %d\n' % (status_left, status_middle, status_right))
```


Tracking module

```
# Main program execution
if __name__ == '__main__':
    try:
        setup()  # Initialize GPIO settings

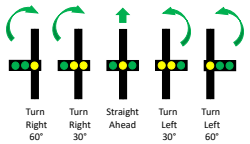
        # Continuously read sensor values and print them until interrupted
        while True:
            run()              # Read and display sensor values
            time.sleep(0.2)    # Wait for 0.2 seconds before the next reading

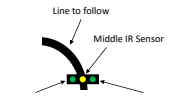
    # Handle keyboard interruption (e.g., Ctrl+C) to safely exit
    except KeyboardInterrupt:
        print('Execution stopped.')    # Print exit message
        GPIO.cleanup()                 # Clean up GPIO settings to avoid issues on next run
```

`run()`: This function reads the status of each sensor and prints their values, where 1 means no line detected and 0 means a line is detected.

Tracking module – Task to be done – TSK05

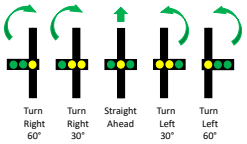
- **Servo Control Function**: Develop a function to control the servo motor responsible for the robot's front wheel steering according to the sensor readings
  - Left Sensor: If the left IR sensor detects the white line, steer the front wheels slightly left.
  - Middle Sensor: If the middle IR sensor detects the line, keep the wheels straight.
  - Right Sensor: If the right IR sensor detects the line, steer slightly right.

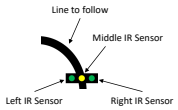




Tracking module – Task to be done – TSK06

- **Sensor Interrupts and Callback**: Configure each IR sensor as an interrupt source.
  - When a sensor detects the white line, an interrupt triggers a callback function to display the status of the sensors and adjust the servo motor control as needed.
  - This real-time feedback allows continuous monitoring of which sensor(s) detect the line, enabling precise and responsive steering adjustments.





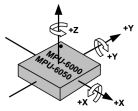
Presentation outline

1. Introduction to Python Programming
2. Introduction to Raspberry Pi and Robot HAT
3. Introduction to Linux Command Line Interface (CLI)
4. Configuring and Integrating Robot Peripherals (Part 1)
   - Controlling LED (Warm color and WS2812)
   - Controlling DC motors
5. Advanced Programming Techniques
6. Configuring and Integrating Robot Peripherals (Part 2)
   - Multi-Threaded Control of Warm color LED
   - Controlling Servo motors
   - Interfacing with the Ultrasonic module
   - Interfacing with the Tracking module
   - **Reading data from the MPU6050 module**
   - Displaying Text on the OLED screen

MPU6050 module

- The MPU6050 is an Inertial Measurement Unit (IMU) sensor that combines a 3-axis gyroscope and a 3-axis accelerometer.
- It measures both the **angular velocity** (gyroscope) and **linear acceleration** (accelerometer) along the X, Y, and Z axes.
- The MPU6050 is commonly used to determine orientation and detect motion in embedded systems and robotics.





MPU6050 module – Principles of Operation

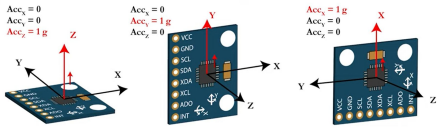
- **Angular Velocity**: Angular velocity is the rate at which an object rotates or spins around an axis.
  - It is measured in terms of the angle covered per unit of time, typically in degrees per second (°/s) or radians per second (rad/s).
  - In the context of sensors like the **MPU6050**, the gyroscope provides angular velocity data for the X, Y, and Z axes, representing the rotation rate around each axis.
- **Gyroscope**: Measures the rate of rotation around each axis. It helps detect angular movement, making it useful for tracking rotational motion.

MPU6050 module – Principles of Operation

- **Linear Acceleration**: Linear acceleration is the rate of change of velocity of an object moving in a straight line.
  - It measures how quickly the speed of an object changes and is typically expressed in meters per second squared (m/s²).
  - In sensors like the **MPU6050**, the accelerometer provides linear acceleration data for the X, Y, and Z axes, showing the force experienced by the sensor in each direction.
- **Accelerometer**: Measures the acceleration due to gravity along each axis. By measuring the tilt (angle relative to the ground) along the X and Y axes, you can determine the orientation of the device.

MPU6050 module – **Tilt** angles

- When the sensor is at rest, the accelerometer senses gravity (usually around 9.81 m/s²).
- The output on each axis (X, Y, Z) reflects the projection of gravitational force on that axis.
- When tilted, the components of gravity detected along the axes change, which can be used to calculate tilt angles (roll and pitch).

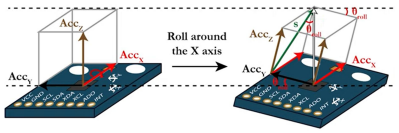


MPU6050 module – **Tilt** angles

- **Tilt** angles are the angles calculated using the accelerometer data to determine the orientation of the sensor relative to the Earth's surface. These angles are usually expressed as **roll** and **pitch**:
  - **Roll (Tilt Around the X-Axis)**: The angle by which the sensor is tilted to the left or right (sideways tilt).
  - **Pitch (Tilt Around the Y-Axis)**: The angle by which the sensor is tilted forward or backward (front-to-back tilt).
- Tilt angles are important for applications in robotics, drone stabilization, and motion tracking to understand the orientation of the device in 3D space.

MPU6050 module – **Tilt** angles

- To calculate angles, you can use **accelerometer** data to determine the tilt angle relative to gravity and **gyroscope** data to measure the rate of angular change.
  - Calculating the ***Roll*** angle in radian:



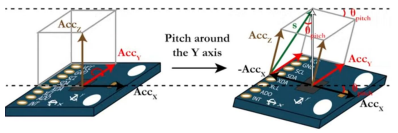
$$\tan\theta_{roll} = \frac{Acc_Y}{s}$$

$$s^2 = (Acc_Z)^2 + (Acc_X)^2$$

$$\theta_{roll} = \operatorname{atan}\left(\frac{Acc_Y}{\sqrt{(Acc_X)^2 + (Acc_Z)^2}}\right)$$

MPU6050 module – **Tilt** angles

- To calculate angles, you can use **accelerometer** data to determine the tilt angle relative to gravity and **gyroscope** data to measure the rate of angular change.
  - Calculating the ***pitch*** angle in radian:



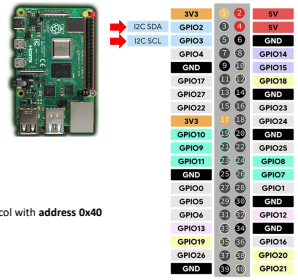
$$\tan\theta_{pitch} = \frac{-Acc_X}{s}$$

$$s^2 = (Acc_Z)^2 + (Acc_Y)^2$$

$$\theta_{pitch} = \operatorname{atan}\left(\frac{-Acc_X}{\sqrt{(Acc_Y)^2 + (Acc_Z)^2}}\right)$$

Raspberry Pi – Robot peripherals interfacing through GPIO

- **Tracking module:**
  - **Left line IR sensor** is connected to **GPIO 19**
  - **Middle line IR sensor** is connected to **GPIO 16**
  - **Right line IR sensor** is connected to **GPIO 20**
- **Ultrasonic sensor:**
  - **Trig** pin of the ultrasonic sensor: **GPIO 11**
  - **Echo** pin of the ultrasonic sensor: **GPIO 8**
- **I2C COM** Protocol (GPIO 2 and GPIO 3)
  - **Servo motors are controlled by the PCA9685 chip:**
    Interfacing to Raspberry Pi is done using I2C COM protocol with **address 0x40**
  - **MPU6050** uses I2C COM protocol with **address 0x68**
  - **OLED screen** uses I2C COM protocol with **address 0x3C**



MPU6050 module – **Tilt** angles

```
# Check the provided Python file:
# MPU6050_v8.py

from mpu6050 import mpu6050  # Import the mpu6050 module to interface with the MPU6050 sensor
import time                  # Import time module for adding delays
import math                  # Import math module for angle calculations

# Initialize the MPU6050 sensor with the I2C address 0x68
sensor = mpu6050(0x68)

def calculate_angles_mpu6050():
    # Calculates and prints roll and pitch angles using accelerometer data.
    accel_x = 0  # Variable to accumulate X-axis acceleration data
    accel_y = 0  # Variable to accumulate Y-axis acceleration data
    accel_z = 0  # Variable to accumulate Z-axis acceleration data
```


MPU6050 module – **Tilt** angles

```
    # Collect 10 samples of accelerometer data to smooth out noise by averaging the results
    for i in range(0, 10):
        accelerometer_data = sensor.get_accel_data()  # Read accelerometer data from the sensor
        accel_x += accelerometer_data['x']            # Accumulate X-axis data (with potential calibration)
        accel_y += accelerometer_data['y'] + 0.200    # Accumulate Y-axis data with a calibration offset
        accel_z += accelerometer_data['z'] - 1.940    # Accumulate Z-axis data with a calibration offset

    # Print averaged acceleration data for X, Y, and Z axes
    print('\nX=%.2f, Y=%.2f, Z=%.2f' % (accel_x/10, accel_y/10, accel_z/10))

    # Calculate the roll angle using the arctangent function and convert to degrees
    roll = math.atan2(accel_y, math.sqrt(accel_x**2 + accel_z**2)) * 180 / math.pi

    # Calculate the pitch angle using the arctangent function and convert to degrees
    pitch = math.atan2(-accel_x, math.sqrt(accel_y**2 + accel_z**2)) * 180 / math.pi
```


MPU6050 module – **Tilt** angles

```
    # Print the calculated roll and pitch angles
    print(f"\nRoll: {roll:.2f}, Pitch: {pitch:.2f}")

if __name__ == "__main__":
    try:
        while True:
            calculate_angles_mpu6050()  # Call the function to calculate and print roll and pitch
            time.sleep(0.5)             # Wait for 0.5 seconds before the next reading
    except KeyboardInterrupt:
        # Handle program interruption gracefully
        print("\nProgram interrupted.")
```


MPU6050 module – Tasks to be done – TSK07

- Modify the provided function to let the robot detects tilting and responds accordingly:
  - Continuously read acceleration and orientation.
  - Stop or change robot behavior if tilt exceeds a threshold.
  - If the robot tilts more than 15° forward, stop and display a warning.

Presentation outline

1. Introduction to Python Programming
2. Introduction to Raspberry Pi and Robot HAT
3. Introduction to Linux Command Line Interface (CLI)
4. Configuring and Integrating Robot Peripherals (Part 1)
   - Controlling LED (Warm color and WS2812)
   - Controlling DC motors
5. Advanced Programming Techniques
6. Configuring and Integrating Robot Peripherals (Part 2)
   - Multi-Threaded Control of Warm color LED
   - Controlling Servo motors
   - Interfacing with the Ultrasonic module
   - Interfacing with the Tracking module
   - Reading data from the MPU6050 module
   - **Displaying Text on the OLED screen**

OLED screen

- An OLED module (Organic Light Emitting Diode) is a type of display that emits light when an electric current passes through it.
  - OLEDs are known for their high contrast, bright colors, and wide viewing angles, making them ideal for use in various embedded systems and robotic projects.
  - An OLED module can be used to display information such as sensor readings, status updates, or user interface elements.



OLED screen

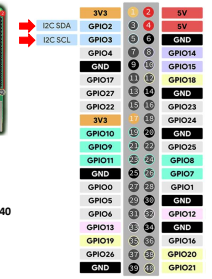
- The OLED display typically operates between 3.3V and 5V with a low power consumption of around 10-20 mA, slightly increasing at maximum brightness.
- It interfaces via the I2C protocol using the default address 0x3C, and communicates over the SDA (data) and SCL (clock) lines at speeds up to 400 kHz.
- Configuration of display settings and screen updates are handled through I2C commands, making it efficient and ideal for embedded applications due to its low power requirements and ease of integration.



Raspberry Pi – Robot peripherals interfacing through GPIO

- **Tracking module:**
  - **Left line IR sensor** is connected to **GPIO 19**
  - **Middle line IR sensor** is connected to **GPIO 16**
  - **Right line IR sensor** is connected to **GPIO 20**
- **Ultrasonic sensor:**
  - **Trig** pin of the ultrasonic sensor: **GPIO 11**
  - **Echo** pin of the ultrasonic sensor: **GPIO 8**
- **I2C COM** Protocol (GPIO 2 and GPIO 3)
  - **Servo motors are controlled by the PCA9685 chip:**
    Interfacing to Raspberry Pi is done using I2C COM protocol with **address 0x40**
  - **MPU6050** uses I2C COM protocol with **address 0x68**
  - **OLED screen** uses I2C COM protocol with **address 0x3C**





OLED screen

- To manage the OLED display, we provide the class `OLED_Display` that includes:
  - `__init__()`: Sets up I2C communication, initializes the display, and prepares empty content lines.
  - `display_text()`: Draws the current content of screen_content on the OLED using the canvas.
  - `update_line(line_number, text)`: Updates a specified line with new text and immediately refreshes the display.
  - `set_all_lines(lines)`: Updates all lines with new content at once, then refreshes the display.

- This structure allows you to initialize, update, and display text on the OLED easily, making the code modular and readable.

OLED screen

```
def __init__(self, num_lines=6, i2c_port=1, i2c_address=0x3C, rotate=0):
    # Initialize the OLED display with specified settings or default values.
    try:
        # Set up the I2C connection to the OLED display.
        # 'port' specifies the I2C port (1 is the default on Raspberry Pi).
        # 'address' is the I2C address of the OLED display, often 0x3C for small SSD1306 displays.
        self.serial = i2c(port=i2c_port, address=i2c_address)

        # Initialize the SSD1306 OLED display with the specified rotation.
        # 'rotate' allows setting the orientation of the display (0, 90, 180, 270 degrees).
        self.device = ssd1306(self.serial, rotate=rotate)

        # Create a list to store text content for each line on the display.
        # 'num_lines' specifies the number of lines that the display will handle.
        # Each line is initially set to an empty string.
        self.screen_content = [""] * num_lines

    # Handle exceptions if the OLED is not connected or an error occurs during initialization.
    except:
        print('OLED disconnected or not found:')
```

```
# Check the provided Python file:
# OLED_vS.py
```


OLED screen

- **I2C Transmission**: Data is only transmitted to the SSD1306 display once, when exiting the `with canvas(self.device)` block, minimizing display flicker.
- **Efficient Display Update**: This approach leverages the I2C protocol's ability to send buffered data in one go, making the display update smooth and power-efficient.

```
def display_text(self):
    #Displays the current content stored in the screen_content list.

    # Open a drawing canvas on the OLED display (data drawn in memory first)
    with canvas(self.device) as draw:

        # Loop through each line of text in the screen_content list
        for i, text in enumerate(self.screen_content):

            # Draw each line of text 10 pixels apart, in white color
            draw.text((0, i * 10), text, fill="white")

    # When the 'with' block exits, the entire canvas is sent to the OLED over I2C
    # This updates the display in a single transmission, reducing flicker
```


OLED screen

```
def update_line(self, line_number, text):
    #Updates a specific line on the screen and refreshes the display.

    # Check if the line number is within valid range
    if 0 <= line_number < len(self.screen_content):

        # Update the specified line in screen_content with the new text
        self.screen_content[line_number] = text

        # Refresh the display to show the updated content
        self.display_text()
    else:
        # Print an error if the line number is invalid
        print(f"Invalid line number: {line_number}. Please choose between 0 and
                                                    {len(self.screen_content) - 1}.")
```


OLED screen

```
# Initialize the OLED display object
screen = OLED_vS.OLED_Display()

# Define the initial content for each line on the display
init_lines = [
    "Main program start",      # Line 1
    "Monitoring data...",      # Line 2
    "Waiting for input...",    # Line 3
    "Line 4: Active",          # Line 4
    "Line 5: Status OK",       # Line 5
    "Line 6: Running"          # Line 6
]

# Set up the initial display with the specified content for all lines
screen.set_all_lines(init_lines)
```


OLED screen – Tasks to be done – TSK08

- Update the previously implemented tasks to display the following information on the OLED screen:
  - Distance to obstacle as detected by the ultrasonic sensor.
  - Tilt angles (Roll/Pitch) calculated from accelerometer data.
  - Display a warning when needed.